



Unsupervised feature based algorithms for time series extrinsic regression

David Guijo-Rubio^{1,2} · Matthew Middlehurst³ · Guilherme Arcencio⁴ ·
Diego Furtado Silva⁵ · Anthony Bagnall^{2,3}

Received: 6 November 2023 / Accepted: 17 April 2024 / Published online: 19 May 2024
© The Author(s) 2024

Abstract

Time Series Extrinsic Regression (TSER) involves using a set of training time series to form a predictive model of a continuous response variable that is not directly related to the regressor series. The TSER archive for comparing algorithms was released in 2022 with 19 problems. We increase the size of this archive to 63 problems and reproduce the previous comparison of baseline algorithms. We then extend the comparison to include a wider range of standard regressors and the latest versions of TSER models used in the previous study. We show that none of the previously evaluated regressors can outperform a regression adaptation of a standard classifier, rotation forest. We introduce two new TSER algorithms developed from related work in time series classification. FreshPRINCE is a pipeline estimator consisting of a transform into a wide range of summary features followed by a rotation forest regressor. DrCIF is a tree ensemble that creates features from summary statistics over random intervals. Our study demonstrates that both algorithms, along with InceptionTime, exhibit significantly better performance compared to the other 18 regressors tested. More importantly, DrCIF is the only one that significantly outperforms a standard rotation forest regressor.

Keywords Time series regression · Extrinsic regression · Interval ensembles · Unsupervised feature extraction

1 Introduction

Time series analysis is a popular topic in machine learning and data mining research. Thousands of research papers in this field have been published in the last decade. Various algorithms have been proposed for disparate tasks across a wide range of applications. The main reason for this development is the increased ability to store

Responsible editor: Eamonn Keogh.

Extended author information available on the last page of the article

data over time and the spread of cheap sensor technology to most fields of science. For example, solar panels depend on sensors to maximise their potential (e.g. to tilt the solar panel so that the sun shines directly on it) and hospitals routinely record and store patient data such as vital signs. This vast wealth of data offers great potential for data mining.

Two of the most researched time series machine learning/analysis tasks are classification (Bagnall et al. 2017; Middlehurst et al. 2023; Ruiz et al. 2021) and forecasting (Makridakis et al. 2008). Time Series Classification (TSC) involves building a predictive model from (possibly multivariate) time series for a categorical target variable. TSC differs from standard classification in that the discriminatory features are often in the shape of the series or the autocorrelation. Forecasting consists of predicting (usually numeric) values based on past observations. Forecasting is usually approached through a model-based algorithm (e.g., autoregressive or exponential smoothing) or by reducing the forecasting problem to a regression problem through a sliding window then using deep learning or a global model such as XGBoost.

Tan et al. (2021) formally specified a related, but distinct, type of time series regression problem: Time Series **Extrinsic** Regression (TSER). Rather than being derived from a forecasting problem, TSER involves a predictive model built on time series to predict a real-valued variable distinct from the training input series. For example, Fig. 1 shows soil spectrograms which can be used to estimate the potassium concentration. Ground truth is found through expensive lab based experiments that take some time. Spectrograms (ordered data series we treat as time series) are cheap to obtain and the data can be collected in any environment. An accurate regressor from spectrogram to concentration would make land and crop management more efficient. A TSER example already in the archive is shown in Fig. 2. Each multivariate time series, comprising an electrocardiogram (ECG) and a photoplethysmogram (PPG), can be used for heart rate estimation.

TSER is related to TSC as traditional regression is to classification: the only difference is that the target variable is real-valued rather than categorical. The distinction *extrinsic* is required because of the prevalence of the term time series regression in the forecasting literature to mean *reduce forecasting to regression through a sliding window*.

The first benchmarking work for TSER (Tan et al. 2021) introduced an archive of 19 TSER problems, including four univariate and 15 multivariate datasets. They performed an experimental comparison of the performance of 13 algorithms on these data. The two algorithms adapted from the TSC literature, the RandOm Convolutional Kernel Transform (ROCKET) (Dempster et al. 2020) and the deep learner InceptionTime (Ismail Fawaz et al. 2020), were top-ranked. However, there was no significant difference in Root Mean Square Error (RMSE) between the ten best-performing algorithms, possibly because of the relatively small number of datasets and the conservative nature of the adjustment for multiple tests used. The abstract of Tan et al. (2021) states that “*we show that much research is needed in this field to improve the accuracy of ML models [for TSER]*”.

Despite the paper’s popularity and the identification of a clear need for novel research, there has been little or no progress in addressing this challenge. We have

responded to this call to arms and developed and assessed a range of TSER algorithms. We have proposed new algorithms that are significantly better than the ones evaluated in Tan et al. (2021).

Our starting point to TSER is to adapt TSC algorithms for regression. The ROCKET family of classifiers all involve transformations using randomised convolutions and a pooling operation followed by a linear or ridge classifier. The original ROCKET was converted to TSER by switching the classifier for a ridge regressor. We extend this to consider a more recent ROCKET variant, MultiROCKET (Tan et al. 2022). Deep learning algorithms are also simple to adapt, and we extend the previous study to include a convolutional neural network in addition to an ensemble regression version of InceptionTime (Ismail Fawaz et al. 2020).

An alternative approach to TSC is to use a large number of unsupervised summary features as a transform. A review of a range of alternatives (Middlehurst and Bagnall 2022) found that the Fresh Pipeline with RotatIoN forest Classifier (Fresh-PRINCE) was the best transform pipeline for TSC. The FreshPRINCE uses the Time Series Feature Extraction based on Scalable Hypothesis Tests (TSFresh) (Christ et al. 2018) followed by a Rotation Forest (RotF) classifier (Rodriguez et al. 2006). We implement the FreshPRINCE for TSER.

Interval-based classifiers also extract unsupervised features, but they do so by ensembling pipelines with randomly selected intervals and a fast base classifier. The first interval-based approach for TSC was the Time Series Forest (TSF) (Deng et al. 2013). TSF generates a set of random intervals and concatenates each interval's mean, standard deviation and slope to make a unique feature space for every base classifier. The Canonical Interval Forest (CIF) (Middlehurst et al. 2020a), and the subsequent Diverse Representation Canonical Interval Forest (DrCIF) (Middlehurst et al. 2021), adopt a similar model to the TSF but use different summary features and data representations. CIF uses the Canonical Time Series Characteristics (Catch22) (Lubba et al. 2019) feature set. Details of these transformation-based algorithms and how we have adapted them to TSER are provided in Sect. 3. Implementations of these regressors are available in the *aeon* toolkit¹ Our main contributions can be summarised as follows:

1. We provide 44 new datasets to the TSER archive, including 24 univariate and 20 multivariate datasets, to take the archive to 63 datasets;
2. We extend the study from Tan et al. (2021) on these new data to examine whether the conclusions translate to the larger collection;
3. We implement recently proposed convolutional-based, feature-based, interval-based, and deep learning-based TSC algorithms to TSER;
4. We conduct an extensive experimental study using 21 regressors and demonstrate that feature-based and interval-based regressors, on average, achieve a significantly better RMSE than any other assessed algorithms;

¹ <https://www.aeon-toolkit.org>.

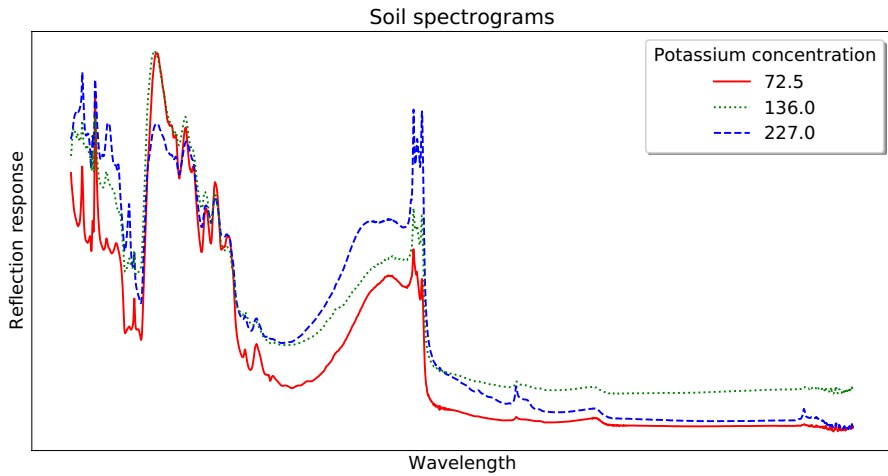


Fig. 1 Three examples of soil spectrograms from the PotassiumConcentration dataset. These spectrograms are used to predict the target variable, the potassium concentration level, the values of which are shown in the legend

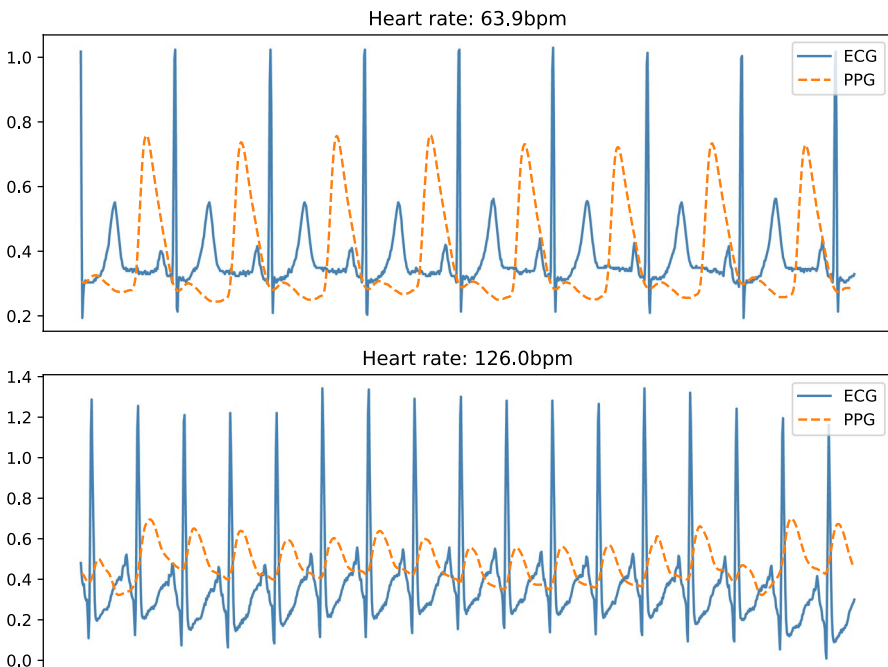


Fig. 2 Two examples of electrocardiograms and photoplethysmograms, with their corresponding heart rate values, from the BIDMC32HR dataset

5. We carry out a comprehensive analysis and an ablative study of the two best proposed approaches: FreshPRINCE and DrCIF.
6. We provide open source implementations of scikit-learn compatible implementations, clear guidance on reproducibility, and detailed results on the associated repository².

The rest of this paper is structured as follows. In Sect. 2, the background and related works are exposed. Sect. 3 describes our new TSER algorithms in detail. In Sect. 4, we give an overview of the new archive and describe our experimental setup. In Sect. 5, experimental results for the 21 approaches applied to the total of 63 datasets are presented. Section 6 looks at these results in more detail. Finally, Sect. 7 summarises our findings and highlights future work.

2 Background and related work

TSER aims to create a mapping function between a time series and a scalar value (Tan et al. 2021). A time series is composed of real-valued ordered observations. Formally, a univariate time series of length m is defined as $\mathbf{x} = \{x_1, x_2, \dots, x_m\}$. A multivariate time series with d channels is specified as $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_d\}$, where $\mathbf{x}_k = \{x_{1,k}, x_{2,k}, \dots, x_{m,k}\}$ and a collection of n time series is denoted $\mathbf{X}$. Hence, $x_{i,j,k}$ represents the j -th observation of the i -th case for the k -th channel. A dataset D is composed of n time series samples and an associated response variable, $D = \{\mathbf{X}, \mathbf{y}\}$, where $\mathbf{y} = \{y_1, y_2, \dots, y_n\}$ are the output continuous values, i. e. the input time series $\mathbf{x}_i$ (univariate) or X_i (multivariate) is associated to the target variable y_i .

A TSER model is a mapping function $\mathcal{T} \rightarrow \mathcal{R}$, where $\mathcal{T}$ is the space of all time series and $\mathcal{R}$ a continuous value. A TSER model is trained on a dataset D_{TRAIN} and evaluated on an independent test dataset D_{TEST} .

TSER shares some similarities with Scalar-on-Function Regression (SoFR) (Goldsmith and Scheipl 2014), a functional regression model where basis models are applied to the series prior to regression, i. e. the goal is to fit a regression model with scalar responses and functional data points as predictors (Reiss et al. 2017). Tan et al. (2021) used two SoFR models in their comparison based on Goldsmith and Scheipl (2014). These were Functional Principal Component Regression (FPCR) and FPCR with B-splines (FPCR-Bs).

Time series forecasting is often reduced to regression through the application of a sliding window to form a collection of time series $\mathbf{S}$ and a forecast horizon specifying how to select the target $\mathbf{y}$. The most common techniques used in Time Series Forecasting Regression (TSFR) include deep learning variants and global models, where channels are concatenated and a standard regressor such as Random Forest or XGBoost is applied.

² https://tsml-eval.readthedocs.io/en/latest/publications/2023/tser_archive_expansion/tser_archive_expansion.html.

Our primary source for TSER (Tan et al. 2021) compared three standard regression algorithms (Support Vector Regressors (SVR) (Drucker et al. 1996), Random Forest (RandF) (Breiman 2001), and eXtreme Gradient Boosting (XGBoost) (Chen and Guestrin 2016)); two k -Nearest Neighbours models using Euclidean and Dynamic Time Warping distances with both one and five neighbours; three deep learning approaches (Fully Convolutional Neural Network (FCN) (Wang et al. 2017), Residual Network (ResNet) (He et al. 2016) and Inception-Time (Ismail Fawaz et al. 2020)); two functional analysis approaches (FPCR and FPCR with B-splines (Goldsmith and Scheipl 2014)); and ROCKET (Dempster et al. 2020). The standard regressors adopt the approach of global forecasting regressors: time series are flattened into a vector concatenating all the channels. Hence, a multivariate time series of length m and d channels is converted into a single vector of length $m \times d$. Subsequently, there have been very few algorithmic advances for TSER. Most novel developments are domain specific and not aimed at TSER as a whole. Among them, a Linear Space State Layers (LSSL) model (Gu et al. 2021) has been tested on three TSER datasets, achieving low error metrics. Another state-space model, Liquid-S4 (Hasani et al. 2022) has also been evaluated on those three datasets, and claims better results. An architecture based on Graph Neural Networks called TISER-GCN (Bloemheuvel et al. 2022) has been applied to seismic data as an extrinsic regression task. In a similar context, Siddiquee et al. (2022) introduces Septor, a hierarchical neural network model developed to estimate the depth of seismic events from waveform data, i.e. a domain-specific extrinsic regression task. ROCKET-XGBoost (Bayani 2022) has been the only novel algorithm evaluated on the 19 TSER archive datasets, but it offered no significant improvement over the algorithms evaluated in Tan et al. (2021).

2.1 Time series classification (TSC) algorithms

There are a plethora of algorithms for TSC that have been compared in reproducible comparative studies (Bagnall et al. 2017; Middlehurst et al. 2023; Ruiz et al. 2021). Broadly, algorithms can be grouped by how they extract and learn from temporal patterns in the time series. We provide a very brief overview with a focus on how classifiers have been or could be adapted to TSER.

Distance-based classifiers use a distance function in conjunction with an algorithm such as Nearest Neighbour (NN) classifier. The two most commonly used distance functions are Euclidean distance and Dynamic Time Warping (DTW). A NN classifier can trivially be adapted for regression by averaging over the target variable of the NN. For multivariate data, using terminology presented in Shokoohi-Yekta et al. (2017), DTW can either be independent (find DTW distance on each channel separately then sum the values) or dependent (use all channels in the point wise distance calculation).

Feature-based algorithms transform series into features using unsupervised descriptive statistics, then complete the pipeline with a classifier trained on the new feature set.

Interval-based classifiers are an extension of feature-based pipeline classifiers where rather than form summary features over the whole series, they concatenate features found over different intervals. They then form an ensemble over different randomised intervals rather than use a single estimator. Together, we group feature-based and interval-based approaches together as unsupervised feature-based classifiers. Adapting these algorithms to TSER is our primary research goal, so we cover this topic in more detail in Sect. 3.

Kernel/convolution-based models find convolutions from the space of all possible subseries and use them to create features through a form of pooling operation. The most popular approach, ROCKET (Dempster et al. 2020) generates random convolutions and is used in conjunction with a ridge classifier in a pipeline. It was adapted for TSER by simply changing the ridge classifier for a ridge regressor (Tan et al. 2021). More recently, MultiROCKET (Tan et al. 2022) was proposed as an improved version of ROCKET. ROCKET uses two pooling operations to generate features: max pooling and the percentage of positive values. MultiROCKET adds three new pooling operations: mean of positive values, mean of indices of positive values, and longest stretch of positive values. It also extracts features from first order differences in addition to the raw data.

Deep learning continues to be popular for TSC (Ismail Fawaz et al. 2019), although to our knowledge InceptionTime (Ismail Fawaz et al. 2020) is still the best performing deep learner. The study in Tan et al. (2021) used Residual Networks (ResNet), Fully Convolutional Neural Network (FCN), and InceptionTime. The original InceptionTime paper (Ismail Fawaz et al. 2020) proposed an ensemble of five InceptionTime classifiers to obtain the final results. However, Tan et al. (2021) used a single InceptionTime model for TSER. We evaluate both a single InceptionTime (Inception) and an InceptionTime Ensemble (InceptionE) faithful to the TSC version for TSER. We also evaluate the Convolutional Neural Network (CNN) regressor based on the classifier described in Zhao et al. (2017).

Shapelet-based approaches (Bostrom and Bagnall 2017; Ye and Keogh 2011) base classification on the presence of selected phase-independent subseries found from the training data. For classification, shapelets are assessed with a supervised measure such as information gain. Furthermore, the most accurate shapelet-based approaches (Middlehurst et al. 2021) evaluate shapelets with a one vs many approach and balance the search procedure between classes to improve diversity. Adapting shapelets for TSER requires significant internal changes and design decisions, being beyond the scope of this paper.

Dictionary based algorithms use a bag of words-like approach to base classification on the number of occurrences of approximated subseries (patterns). The most successful dictionary-based classifiers (Schäfer 2015; Schäfer and Leser 2023; Middlehurst et al. 2020b) involve a degree of supervised selection using accuracy for filtering/weighting or feature selection and their adaptation for TSER is also beyond the scope of this paper.

3 Unsupervised feature-based regressors

Approaches which extract features from time series in an unsupervised process have been shown to perform well in classification scenarios. ROCKET (Dempster et al. 2020) and CIF (Middlehurst et al. 2020a) perform as well as or better than single representation approaches such as shapelet or dictionary algorithms. These algorithms also have the benefit of lower complexity, essentially consisting of transform to classifier pipelines or an ensemble of pipelines. ROCKET and CIF (Middlehurst et al. 2020a) were also top ranked for Multivariate TSC (MTSC) in a recent survey (Ruiz et al. 2021).

We describe the features extracted and our adaptations for two additional algorithms based on unsupervised transformations. The first is the FreshPRINCE (Middlehurst and Bagnall 2022), a pipeline using the TSFresh (Christ et al. 2018) feature set. The second is DrCIF (Middlehurst et al. 2021), an interval-based ensemble.

3.1 FreshPRINCE

FreshPRINCE is a pipeline algorithm for regression with two components: the TSFresh feature extraction algorithm that transforms the input time series into a feature vector, and then a Rotation Forest (RotF) (Rodriguez et al. 2006) estimator that builds a model and makes target predictions. The structure of a generic pipeline algorithm for TSER is displayed in Fig. 3. TSFresh (Christ et al. 2018) is a collection of just under 800 features that can be extracted from time series data. While the features can be used on their own, a feature selection method called Fresh is provided to remove irrelevant features. FreshPRINCE does not make use of this feature selection. It keeps all the transformation process unsupervised and allows the RotF to decide the utility of features. TSFresh is generally popular within the data science community, and has shown to perform better than other unsupervised transformation pipelines on classification problems as part of FreshPRINCE (Middlehurst and Bagnall 2022).

RotF is an ensemble of tree classifiers which has been shown to accurately make predictions for problems where the attributes are continuous (Bagnall et al. 2018). The classifier has been used as a benchmark and as a part of other pipeline classifiers in TSC (Bagnall et al. 2017; Middlehurst et al. 2021), and performed better than a ridge classifier and XGBoost (Chen and Guestrin 2016) when paired with unsupervised transforms for TSC (Middlehurst and Bagnall 2022). Full descriptions of the RotF algorithm are available in Rodriguez et al. (2006) and Bagnall et al. (2018). RotF is easily adaptable for regression: the implementation we developed removes class subsampling (Pardo et al. 2013), replaces the C4.5 decision tree with a Classification and Regression Tree (CART) (Breiman 2017), and averages the target predictions for each tree in the forest. The full TSFresh transformation and altered RotF make up our FreshPRINCE adaptation for TSER.

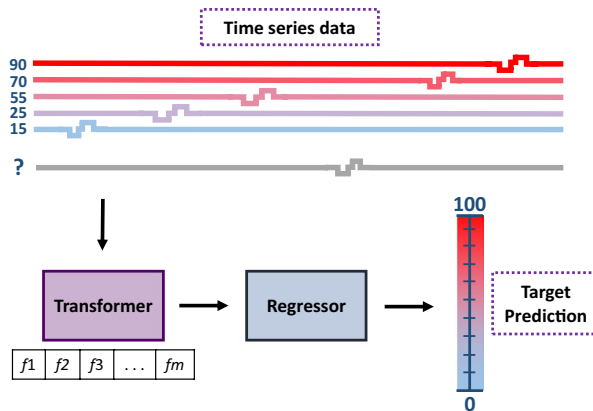


Fig. 3 A diagram visualising a simple transformation pipeline for TSER. A transformation will convert the series into a usable feature vector for a regression algorithm

3.2 DrCIF

Interval-based techniques select phase-dependent intervals of fixed offsets from which to extract summary features. These intervals share their position for all time series, with the aim of discovering discriminatory features from particular locations in time. Most interval techniques take the form of a forest of decision trees, using different intervals to achieve diversity in the ensemble. While some interval forests do make use of supervised feature extraction (Cabello et al. 2020), TSF (Deng et al. 2013) and DrCIF (Middlehurst et al. 2021) are completely unsupervised in their method for selecting intervals and extracting features from said intervals. All that we change for TSER from the classification implementation is a swapping of the tree algorithm used. TSF can be adapted for the regression task in the same way.

From a series of length m , there are $m(m-1)/2$ possible intervals when considering all interval lengths and positions. Even at small series lengths, it is unfeasible to extract features from or evaluate all possible intervals. To solve the issue of which intervals from this pool to select, DrCIF uses a random forest based approach. An ensemble of CART regressors is formed, built on the output of different random interval transformations. Algorithm 1 describes the full build process for DrCIF. The transformation has three steps. First, the base time series is split into three series representations: the original time series, the first order differences of the series, and the periodogram of the series (characterised in line 3 of Algorithm 1). The differences and periodogram series-to-series transformations have shown to provide useful information in classification approaches (Flynn et al. 2019; Cabello et al. 2020; Tan et al. 2022; Keogh and Paz-zani 2001). Then, a different transform is created for each base regressor. First, a pool of a features is selected from a candidate pool of 29 features (line 6). DrCIF makes use of the CAnonical Time series CHAracteristics (Catch22) (Lubba et al. 2019). Catch22

is a diverse set of 22 features filtered from the 7000+ available in the Highly Comparative Time Series Analysis (HCTSA) toolbox (Fulcher and Jones 2017). The Catch22 features were selected for use on normalised data, but we do not make that assumption. Hence, seven additional summary statistics are also candidates: the mean, standard-deviation, slope, median, interquartile range, min, and max. Then, for each data representation, a set of k random intervals are selected (lines 10–13), and the a unsupervised features are calculated and concatenated from a randomly selected channel (lines 13–15). Finally, a CART tree is trained on the feature set unique to each ensemble member. Figure 4 visualises the transformation (upper figure) and ensemble (lower figure) process for DrCIF. Predictions for new cases are found by averaging the predictions of the base regressors.

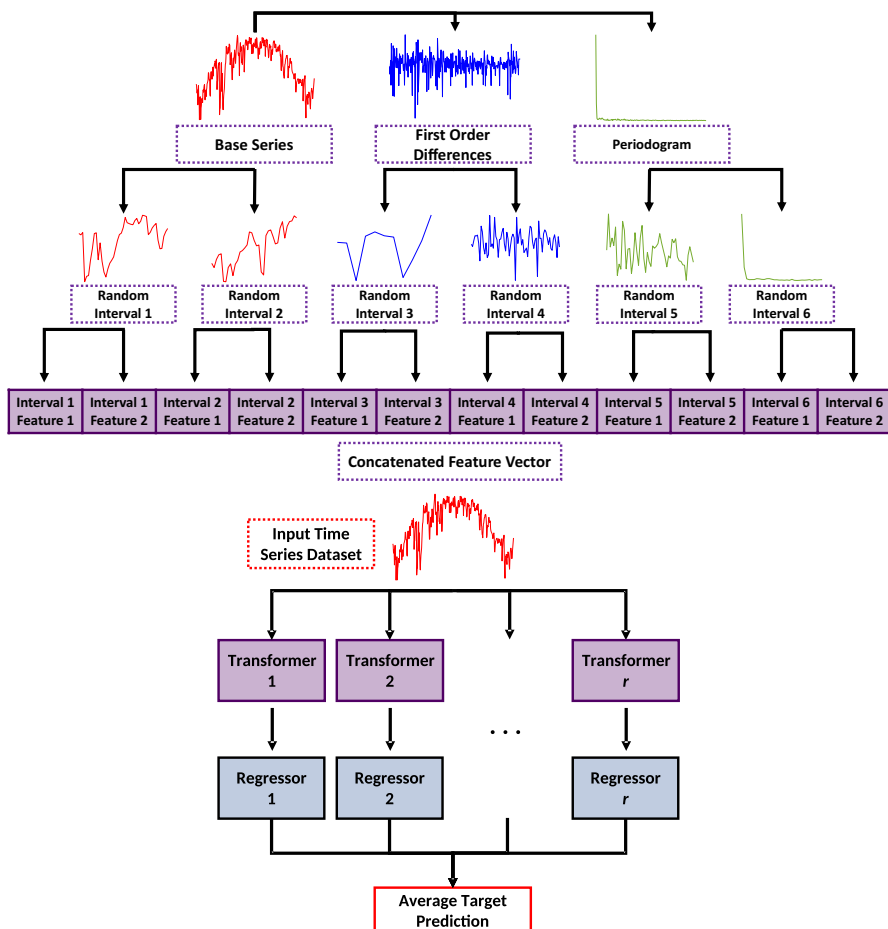


Fig. 4 Diagrams visualising the transformation (top) and ensemble structure (bottom) of DrCIF

Algorithm 1 DrCIF(A list of n cases of length m with d channels, $T = (X, y)$)

Require: the number of trees: r ; the number of intervals per representation for each tree: k ; and the number of attributes subsampled per tree: a (default $r = 500$, $k = 4 + (\sqrt{d}\sqrt{rm})/3$, and $a = 10$) { where rm is the length of a representations series }

- 1: Let $DT = (DT_1 \dots DT_r)$ be the trees in the forest
- 2: Let V be a $3 \times n \times d$ matrix of series with variable length, containing the base series (X), the periodograms of X and first order differences of X
- 3: **for** $i \leftarrow 1$ to r **do**
- 4: $X' \leftarrow []$
- 5: $u \leftarrow$ select a random attribute indices $(u_1, \dots, u_a)$
- 6: **for** $re \leftarrow 1$ to 3 **do**
- 7: $F \leftarrow$ initialise matrix of dimensionality $n \times (ak)$
- 8: **for** $j \leftarrow 1$ to k **do**
- 9: $b = \text{rand}(1, |V_{re}| - 3)$ { interval position }
- 10: $l = \text{rand}(3, |V_{re}|/2)$ { interval length }
- 11: $o = \text{rand}(1, d)$ { interval channel }
- 12: **for** $t \leftarrow 1$ to n **do**
- 13: **for** $c \leftarrow 1$ to a **do**
- 14: $F_{t,a(j-1)+c} \leftarrow \text{summaryStat}(u_c, V_{re,t,o}, b, l)$
- 15: $X' \leftarrow X' + F$ { concatenate feature vectors }
- 16: $DT_i.\text{buildCART}(X', y)$

4 Methodology

We summarise the new problems we have added, the regressors used in experiments, and a description of our experimental method.

The previous version of the TSER archive includes 19 different datasets. We have increased the number of datasets in the archive to 63. There are now 28 univariate problems and 35 multivariate, with number of channels ranging from 2 to 24. Dataset size range from 93 to over 16,000. 70% are used for training, 30% for testing. Series length ranges from 14 to 7500. Nine of the problems contain missing values and two have unequal-length series.

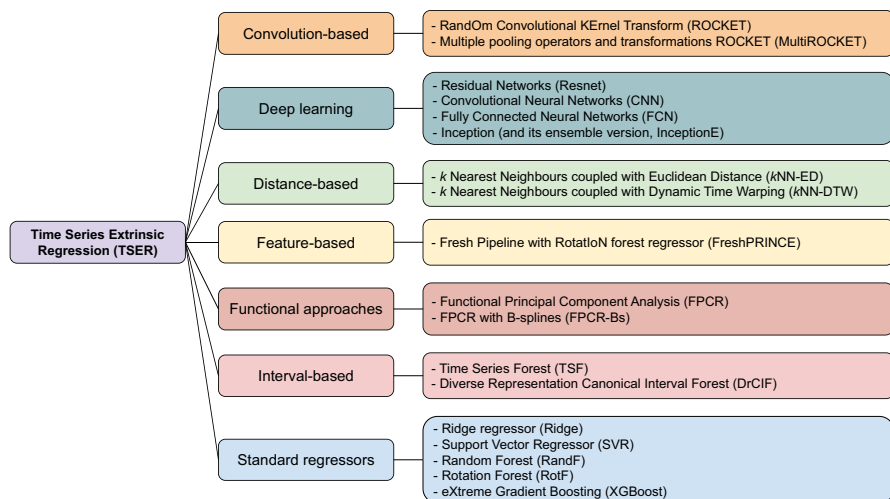
The new datasets have been taken from Kaggle competitions and other archives and repositories/websites associated with applied research. Table 1 summarises the gathered data. More details on the datasets are available in “Appendix A” and on the associated repository. None of the datasets have been normalised. One of the new problems has unequal-length series. For experiments, keeping with the practice in Tan et al. (2021), missing values in the series are linearly interpolated, and unequal-length series are truncated to the minimum length series. Full descriptions, and both unequal and missing values series are available on the associated website.

Table 1 New TSER datasets

Name	Prediction problem (response variable)
<i>Economic analysis</i>	
DailyOilGasPrices	Daily gas price with time series of oil prices ¹ .
<i>Energy monitoring</i>	
Energy building predictors	Estimate the energy consumption of different sorts on buildings ² .
OccupancyDetectionLight	The average hourly occupancy of an office from sensor measurements (Candanedo and Feldheim 2016).
SolarRadiationAndalusia	The average hourly solar radiation from atmospherical measurements ³ .
TetuanEnergyConsumption	The daily average power consumption in Tetouan from atmospherical measurements (Salam and El Hibaoui 2018).
WindTurbinePower	The daily power output of a wind turbine based on time series of torque measurements ⁴ .
<i>Environment monitoring</i>	
AcousticContaminationMadrid	The 1st percentile of sound pressure levels from LAeq ⁵ .
Africa Soil Chemistry	A set of 12 problems derived from the Africa Soil Information Service (AfSIS) Soil Chemistry ⁶ .
BeijingIntAirportPM25Quality	The daily average of PM25 in the Airport of Beijing from atmospherical data (Liang et al. 2015).
DailyTemperatureLatitude	The latitude of a city based on the annual time series of daily temperature ⁷ .
DhakaHourlyAirQuality	The Air Quality Index in Dhaka using localised particular matter time series ⁸ .
MadridPM10Quality	The weekly average of PM10 in the city of Madrid, Spain, from measurements of gases ⁹ .
MetroInterstateTrafficVolume	The daily average traffic volume of a road in the USA from atmospherical variables ¹⁰ .
ParkingBirmingham	The daily occupancy rate from the hourly total number of parked cars (Stolfi et al. 2017).
PrecipitationAndalusia	The yearly average of rainfall on Andalusia, Spain, from meteorological measurements ¹¹ .
SierraNevadaMountainsSnow	The amount of snow based on temperature time series (Osterhuber and Schwartz 2021).
<i>Equipment monitoring</i>	
ElectricMotorTemperature	The temperature of an electric motor based on time series of torque readings ¹² .
LPGas/Methane MonitoringHomeActivity	The liquefied petroleum and methane concentration from gas sensors (Huerta et al. 2016).
GasSensorArray Ethanol/Acetone	The concentrations of two analytes, acetone based on 16 metal-oxide sensors (Ziyatdinov et al. 2015).
WaveTensionData	The tension of a string based on wave elevation time series ¹³ .
<i>Health monitoring</i>	
BarCrawl6min	The transdermal alcohol content by using an accelerometer (Killian et al. 2019).

Table 1 (continued)

Name	Prediction problem (response variable)
Covid19Andalusia	The rate of deceased/contagions people from number of contagions in Andalusia, Spain (Díaz-Lozano et al. 2022).
VentilatorPressure	The pressure of the inspiratory solenoid valve from control input and output of the same valve ¹⁴ .
<i>Sentiment analysis</i>	
Crypto Sentiment	The sentiment of four cryptocurrencies based on the same days hourly price ¹⁵ .
NaturalGasPriceSentiment	Sentiment scores about natural gas prices based on the daily natural gas prices (Ghosh 2022).

¹ <https://perma.cc/DAP4-JC2A>² <https://perma.cc/PA63-7GVU>³ <https://perma.cc/CH23-UVRJ>⁴ <https://perma.cc/8X2R-PRUD>⁵ <https://perma.cc/9V27-BF3F>⁶ <https://perma.cc/TP5Y-KS6M>⁷ <https://perma.cc/3KPY-YHW2>⁸ <https://perma.cc/7865-ZAAD>⁹ <https://perma.cc/ZUS5-E26E>¹⁰ <https://perma.cc/B6FV-SLCG>¹¹ <https://perma.cc/3APP-2L43>¹² <https://perma.cc/A7FG-KFLT>¹³ <https://perma.cc/BAG4-W8SL>¹⁴ <https://perma.cc/RQW7-QH7L>¹⁵ <https://perma.cc/J6LK-99Q5>**Fig. 5** Taxonomy of current literature in TSER. All these regressors have been evaluated in Sect. 5

4.1 Regression algorithms

The full list of the 21 regressors (with associated abbreviation) evaluated in Sect. 5 is presented as a taxonomy in Fig. 5.

Parameter settings for all algorithms are described in “Appendix B”.

4.2 Experimental design

Each dataset is provided with a default train/test split. We repeat every experiment 30 times to mitigate for sampling variation. The first experiment is with the default data test/train split. Subsequent experiments are conducted with data resampled by pooling the train and test and randomly partitioning the data with the same train/test proportions as the original. Performance is measured with the RMSE to conform with Tan et al. (2021). To compare regressors, we first average RMSE over all resamples. We use ranks in all statistical tests. For multiple regressors over multiple datasets we use an adaptation of the critical difference diagram (Demšar 2006), replacing the post-hoc Nemenyi test with a comparison of all classifiers using pairwise Wilcoxon signed-rank tests, and cliques formed using the Holm correction, as recommended in García and Herrera (2008), Benavoli et al. (2016).

5 Results

Our experiments are structured as follows. In Sect. 5.1 we recreate the results presented in Tan et al. (2021) on the original 19 datasets. We then extend the analysis to our larger collection of datasets to test whether the conclusions reached in Tan et al. (2021) generalise to the new archive of 63 problems. In Sect. 5.2 we compare the best performing regressors from the previous experiments to the new algorithms we are proposing, FreshPRINCE and DrCIF. We also include two improvements for regressors used in the previous study and other regressors available in open source toolkits. RMSE results for the best performing regressors on 63 TSER datasets are available in the accompanying website³.

5.1 Recreating results on the 19 TSER datasets

We ran the 13 regressors reported in Tan et al. (2021) on the current 19 datasets in the archive. Figure 6 shows a critical difference diagram of our results alongside the results presented in Tan et al. (2021). Broadly, the ordering of algorithms is the same and the cliques are similar. There are some differences in the ordering, with ROCKET and FCN lower ranked and Inception and ResNet higher in our experiments than the original.

³ https://tsml-eval.readthedocs.io/en/latest/publications/2023/tser_archive_expansion/tser_archive_expansion.html.

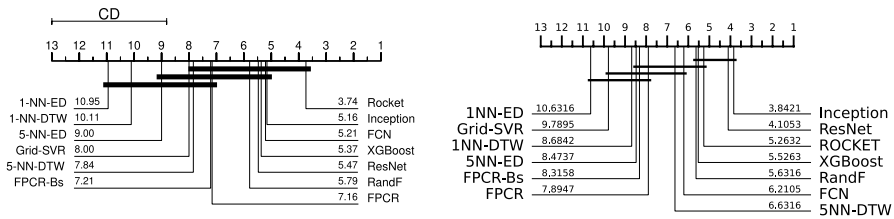


Fig. 6 Reproduction of the RMSE ranks on the original archive (using 19 datasets). Left is the original image from Tan et al. (2021). Right is our recreation using 5 resamples

Differences may be explained by a slight difference in experimental procedure. The experiments in Tan et al. (2021) involved five repetitions on the default train/test split with a different random seed, whereas we resampled the data on each repetition. We did this for consistency with our later experiments. We also have more diverse cliques than observed in Tan et al. (2021). This is because our adjustment for multiple testing is less conservative than the one used in Tan et al. (2021), where a full Bonferroni adjustment is used rather than a Holm correction.

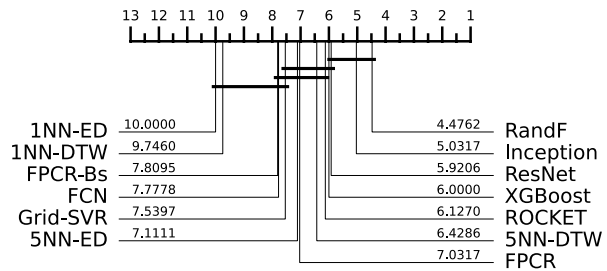
In Fig. 7 we compare the 13 regressors used for Fig. 6 on the larger archive of 63 datasets. For all subsequent experiments we extend the number of resamples on each dataset from five to 30. All resampling is done without replacement and maintains the same train/test sizes of the original datasets. The first resample is always the original train/test split and these resamples are seeded so can be exactly reproduced (see accompanying website for an example). We observe that RandF is now the best, improving significantly on ROCKET, and better in rank than Inception and ResNet, among others. Hence, the time series specific methods previously proposed for TSER are not better than using an off the shelf regressor with concatenated features.

5.2 Benchmarking the new TSER archive

For the next set of experiments, we take the top five algorithms in Fig. 7 and compare these to some alternative adaptations of time series specific algorithms. The good performance of XGBoost and RandF suggests we should not overlook standard classifiers. Rotation Forest (RotF) (Rodriguez et al. 2006) is a classifier that can be easily adapted to regression by simple averaging (Pardo et al. 2013). It has been shown to be particularly effective for problems with all real valued attributes, including time series (Bagnall et al. 2018, 2017). Hence, we include a regression adaptation in this round of experiments. We also add in the standard Ridge regressor for completeness sake. In addition, the open source toolkit `aeon`⁴ includes two regression implementations not previously evaluated in the context of TSER. Time Series Forest (TSF) regressor is an adaptation of the TSF classifier (Deng et al. 2013) and CNNRegressor (CNN) is Convolutional Neural Network based on the version described in Zhao et al. (2017). On further investigation, we found that the results for InceptionTime in Tan et al. (2021) were created with a single InceptionTime

⁴ <https://github.com/aeon-toolkit/aeon>.

Fig. 7 RMSE ranks for 13 regressors used in Tan et al. (2021) on 63 TSER datasets



model (Inception). However, in the original work (Ismail Fawaz et al. 2020), the results supporting InceptionTime as a classifier are found with an ensemble of five InceptionTime models. We include an InceptionTime ensemble model for regression (InceptionE). Furthermore, an improved version of the ROCKET algorithm has been recently published, known as MultiROCKET (Tan et al. 2022). We adapted it to the TSER paradigm accordingly. Finally, we also included the proposed regressors based on unsupervised feature extraction, DrCIF and FreshPRINCE. We provide implementations of all these approaches in the *aeon* toolkit. Figure 8 shows the results of the five best algorithms from experiments presented in Fig. 7 and eight new regressors. We have had to exclude the AustralianRainfall dataset and hence reduce the number of datasets in our study to 62 because we were unable to run experiments with the MultiROCKET regressor: it requires over 600GB memory for this dataset and takes more than 15 days to complete.

Figure 8 shows that DrCIF, FreshPRINCE and InceptionE form the top clique. DrCIF is the top rank regressor, and it is the only algorithm that is significantly better than RotF, the top performing standard approach of all those we have tried. FreshPRINCE achieves the second best averaged rank, though it is not significantly different to several regressors, such as RotF or MultiROCKET. InceptionE is the third best algorithm. InceptionE is often very good: it is top ranked on 13 of the 62 problems, and it is significantly better than a single Inception network, which is top ranked only on one dataset. However, InceptionE also fails spectacularly on many problems. Furthermore, the CNN and Ridge regressors are not competitive with the other eleven algorithms. As expected, MultiROCKET is significantly better than ROCKET. Another interesting feature is that RotF is one of the top performing algorithms, achieving similar results to MultiROCKET, InceptionE and FreshPRINCE. RotF is highly effective with real valued input (Bagnall et al. 2018) and the best performing standard algorithm for TSC (Bagnall et al. 2017), so this is perhaps not surprising. It does well with time series because it removes embedded correlations through randomised PCA transforms on subsets of algorithms. Despite this, the fact that an algorithm for standard regression outperforms a wide range of the deep learning and time series specific approaches is indicative of the scope for improvement in the field of TSER. Thus far, DrCIF is the only regressor that is on average significantly better in terms of RMSE than RotF. In subsequent analyses, the focus is directed towards a subset with the seven top ranked regressors, as shown in Fig. 8.

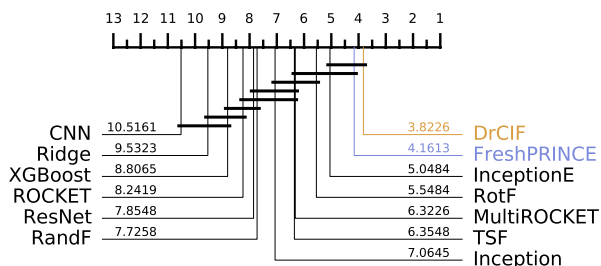
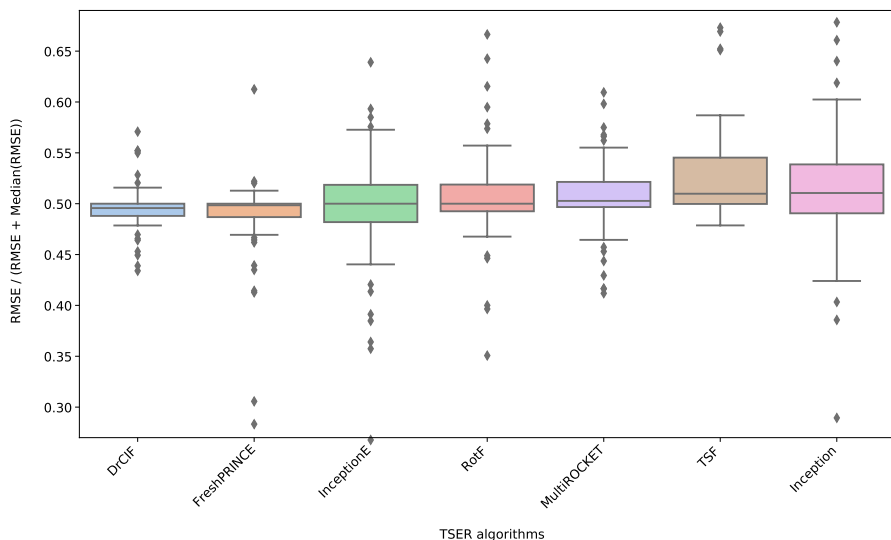
Fig. 8 RMSE ranks for 13 regressors on 62 TSER datasets

Table 2 provides an overview of the mean and Standard Deviation (STD) of the RMSE and the Mean Absolute Error (MAE) for the top seven algorithms. Figure 9 illustrates relative performance of the top seven regressors for RMSE with boxplot. The y-axis for Fig. 9 is the relative deviation of the RMSE, calculated as

Table 2 Mean and Standard Deviation (STD) of RMSE and MAE over all problems. The STD is computed as the average of the STD of the datasets

		DrCIF	Fresh- PRINCE	InceptionE	RotF	Multi- ROCKET	TSF	Inception
RMSE	Mean	659.4550	666.4411	856.9787	675.5686	777.5946	682.1765	852.5311
	STD	162.0449	163.6923	281.7305	162.7472	339.7403	166.9817	291.0105
MAE	Mean	324.2052	309.6700	420.3192	310.5031	396.8601	344.3326	423.7530
	STD	36.4575	46.0679	63.1505	43.5706	72.0082	36.1278	69.3193

Bold values indicate the best performance, italic values indicate the second best

**Fig. 9** Distribution of relative deviation of the Root Mean Square Error (RMSE), calculated as $\frac{RMSE}{RMSE + Median(RMSE)}$ for the top seven regressors (lower values are better)

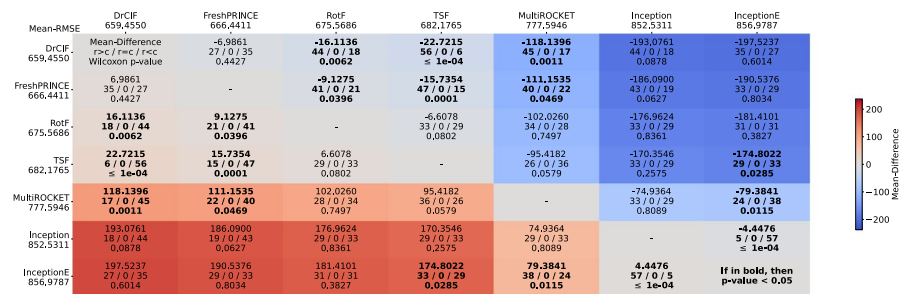


Fig. 10 Summary performance results for the best seven regressors

$\frac{RMSE}{RMSE + Median(RMSE)}$, across all problems. Lower values are better and values below 0.5 indicate performance better than the median algorithm. A tight distribution indicates an algorithm is consistent in its performance relative to other algorithms. When considered together, Table 2 and Fig. 9 highlight the relative performance of these algorithms. DrCIF has the lowest average RMSE closely followed by FreshPRINCE. The latter achieves the lowest average MAE, followed by RotF. The standard deviations demonstrate that the regressors have comparable variability, except for InceptionE, Inception, and MultiROCKET, which have higher variance for both RMSE and MAE. DrCIF again stands out as the most robust and stable, followed by RotF, FreshPRINCE and TSF.

Examining all regressors on the x-axis of Fig. 9, only DrCIF and FreshPRINCE are consistently better than the median performance, and the distribution is tightly coupled. Inception and InceptionE have the widest spread.

Figure 10 summarises the performance of top seven algorithms using a heatmap derived from the average RMSE results. The table was generated with a recently proposed results visualisation tool⁵. These results reinforce our previous observations.

6 Analysis

We explore our results in more detail to better profile the regressors and gain insights into the drivers behind their performance.

6.1 Run time

Figure 11 shows the average rank RMSE against the run time (on a log scale) for the six regressors from Fig. 9. Note that the timings for Inception are not considered reliable as it was executed on a combination of CPU and GPU, potentially leading

⁵ https://github.com/MSD-IRIMAS/Multi_Comparison_Matrix.

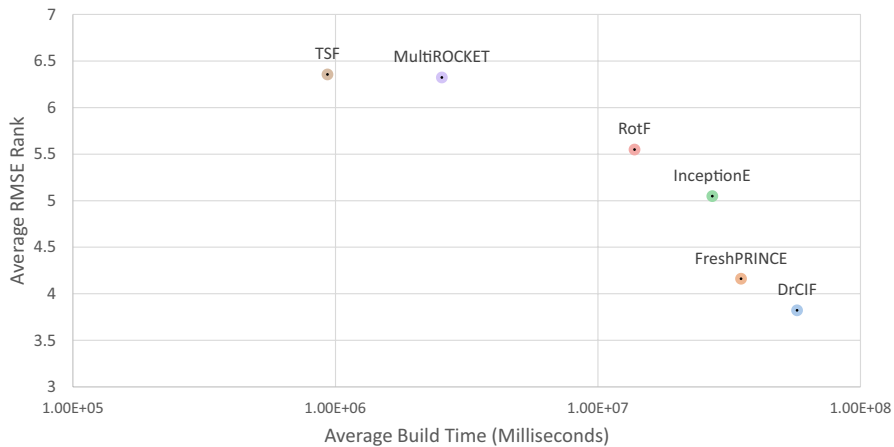


Fig. 11 Run time in milliseconds (log scale average over all datasets) plotted against average rank for RMSE

to confusion. We see a direct trade off between runtime and performance. All algorithms run on a single thread CPU except for InceptionE, which ran on a GPU. This means the graph is very flattering for InceptionE. Even on a GPU it is slower than RotF on a CPU.

6.2 Performance by data characteristics

We break down the performance of regressors by the core characteristics of the data to help gain insights into when different algorithms perform well. We stress this is purely exploratory: the relatively small number of datasets in each category, in brackets, preclude useful significant testing.

Table 3 shows the average rank RMSE when we group problems by the number of training cases. The pattern is that DrCIF and FreshPRINCE are better with a small number of cases, whereas InceptionE is better with larger train set size.

Table 4 shows the average rank RMSE when we group problems by the number of channels. DrCIF is better with univariate problems. FreshPRINCE achieves better results when dealing with multivariate datasets. InceptionE has more potential with multivariate problems with 3 or 4 channels.

Table 5 shows the average rank RMSE when we group problems by the series length. The interval-based DrCIF performs relatively better than FreshPRINCE with long series but worse with shorter series ($\text{length} < 50$). InceptionE achieves a good rank for relatively longer time series ($150 < \text{length} \leq 365$).

Table 3 Average rank RMSE split by number of train cases

	<300 (15)	300–999 (23)	1000–5000 (13)	>5000 (11)
DrCIF	2.93	3.22	4.31	5.73
FreshPRINCE	2.67	4.61	4.62	4.73
InceptionE	7.53	5.48	4.08	1.91
RotF	4.80	5.78	4.77	7.00
MultiROCKET	7.20	4.96	6.62	7.64
TSF	5.13	6.22	7.00	7.55
Inception	8.93	7.39	6.39	4.64

Bold values indicate the best performance for a category, italic values indicate the second best

Table 4 Average rank RMSE split by number of channels

	1 (28)	2 (10)	3 or 4 (13)	≥ 5 (11)
DrCIF	2.86	3.70	5.15	4.82
FreshPRINCE	4.32	3.60	4.85	3.45
InceptionE	5.46	5.60	4.15	4.55
RotF	5.11	5.30	7.54	4.55
MultiROCKET	5.32	7.69	5.32	7.36
TSF	5.07	6.10	7.23	8.82
Inception	7.89	5.77	7.89	6.09

Bold values indicate the best performance for a category, italic values indicate the second best

Finally, we also assessed relative performance for different problem types but did not detect any interesting trends.

6.3 Ablation of FreshPRINCE

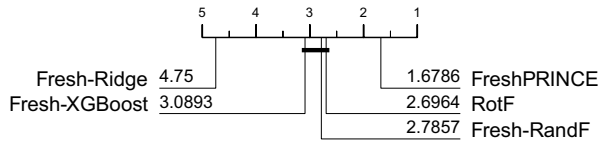
FreshPRINCE is a pipeline of a TSFresh transform and a RotF regressor. We address the question of whether the performance of this regressor is due to the transform, the

Table 5 Average rank RMSE split by series length (there are no problems with length 366–999)

	<50 (13)	50–150 (13)	151–365 (16)	≥1000 (20)
DrCIF	3.39	3.00	5.00	3.70
FreshPRINCE	2.85	3.00	4.13	5.80
InceptionE	6.39	6.69	3.81	4.10
RotF	3.39	4.54	7.38	6.15
MultiROCKET	8.08	8.39	6.19	3.95
TSF	5.77	5.08	7.38	6.75
Inception	8.00	9.39	5.25	6.40

Bold values indicate the best performance for a category, italic values indicate the second best

Fig. 12 RMSE ranks for Fresh-PRINCE, standard RotF and TSFresh (Fresh) with alternative regressors



regressor or both. Figure 12 summarises the performance of FreshPRINCE, RotF on the raw series and TSFresh transform followed by an alternative regressor. It demonstrates that transforming followed by RandF or XGBoost are no better than simply applying RotF to the raw data. We conclude that it is the combination of transform and regressor that give significantly better performance.

There is very little agreement between InceptionE and the other regressors. We believe one reason InceptionE performance is so variable is it sometimes completely fails to find anything useful in a dataset where other models have at least some predictive power. To demonstrate this, we look at the standardised residuals of DrCIF and InceptionE on the BarCrawl6min dataset. The time series are accelerometer data, and the response variable is the transdermal alcohol concentration of the test subjects. The response variable is bounded below by zero. In traditional regression, the analyst might look to transform the response with, for example, a Yeo-Johnson transform (Yeo and Johnson 2000). We are interested in performance over multiple datasets without hand tailored transforms. The RMSE for the default train-test partition for DrCIF is 0.0017 and for InceptionE it is 0.0045. If we plot the predicted response vs the actual response for DrCIF (Fig. 13) we see that DrCIF is making negative predictions for low actual values, underestimating higher values and there seems to be some heteroscedasticity in the residuals. Nevertheless, it has definitely found some relationship between the regressor series and the response. However,

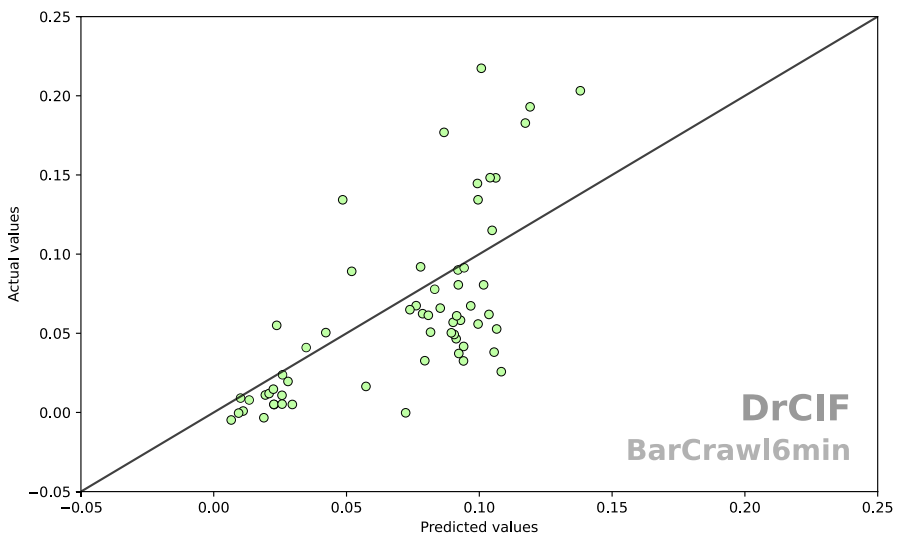


Fig. 13 Scatter plot of predicted vs actual for DrCIF on BarCrawl6min

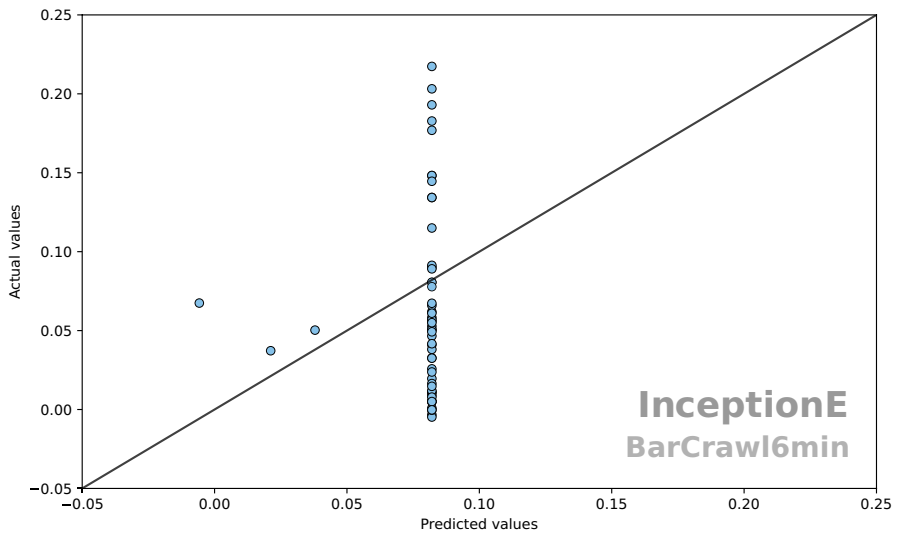


Fig. 14 Scatter plot of predicted vs actual for InceptionE on BarCrawl6min

if we see the same plot for InceptionE (Fig. 14), we see that InceptionE is nearly always predicting the same value of 0.082. It is likely that careful configuration, tuning and transform of the data may improve InceptionE. However, the same is true for DrCIF and freshPRINCE. We are using InceptionE in the way recommended by its creators (Ismail Fawaz et al. 2020).

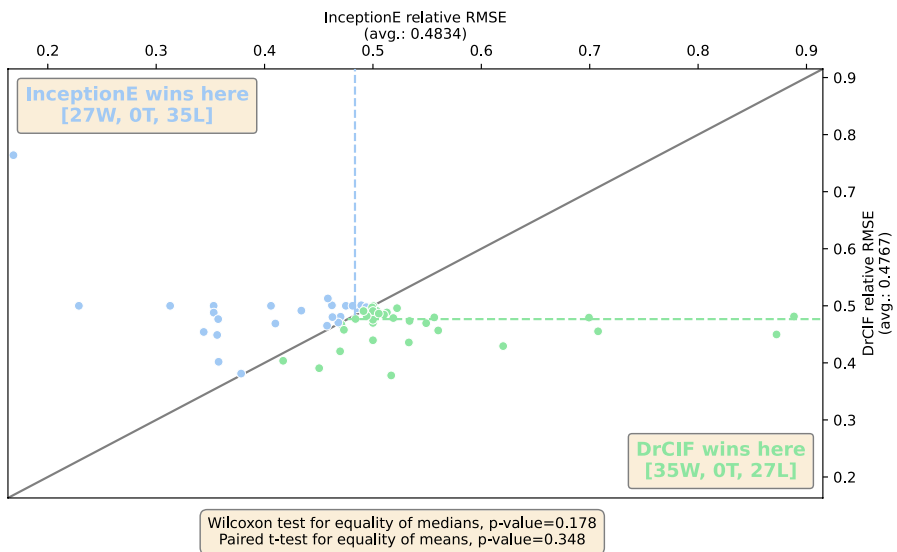


Fig. 15 Scatter plot of relative RMSE for DrCIF and InceptionE

Furthermore, Fig. 15 shows the scatter plot of the relative RMSE used in Fig. 9 for DrCIF vs InceptionE. There is no correlation. For context, for our top four regressors, the strongest correlation is between DrCIF-RotF ($R^2 = 0.27$) and DrCIF-FreshPRINCE are very weakly correlated ($R^2 = 0.13$). This diversity suggests there may be some value in ensembling.

7 Conclusions

We have proposed new algorithms for Time Series Extrinsic Regression (TSER) based on classifiers and conducted an extensive experimental study. We have increased the TSER archive size from 19 to 63, introduced improved versions of regressors used in the previous study and shown our new adaptations of classification algorithms are significantly better than the best alternatives. There are several limitations to this study. The data is not randomly sampled (we have taken problems from where ever we can find them) and some domains may be over represented; we have not tuned any of these regressors (the computation required to tune these algorithms over 63 datasets would be prohibitive); we have not looked at more complex diagnostics of performance such as residual analysis. Nevertheless, we believe we have made a significant contribution to advance the new field of TSER. RotF outperforms all previously assessed regressors, and DrCIF and FreshPRINCE are the only TSER algorithms so far proposed to significantly outperform all standard regression algorithms. We have made all our experiments reproducible by releasing structured source code compatible with standard toolkits, guidance on reproducing experiments, and all of our results. We have hosted our results on the repository ⁶. The datasets and these results can be downloaded directly using the `aeon` toolkit and compared to results for new regressors ⁷. Detailed examples of how to do this are on the repository we use to run experiments⁸.

We believe there is scope for further improvement for algorithms for TSER. Adapting supervised Time Series Classification (TSC) approaches may help further leverage of this popular theme for research. InceptionE is the most promising deep learning approach, and perhaps it can be engineered to avoid the catastrophic failure it tends towards with smaller train set sizes. Heterogeneous ensembles are very successful for TSC, and the diversity in performance between the best performing algorithms suggest this may, with careful adaptation, translate to TSER. We will continue to enhance the repository with more problems and would welcome all donations.

⁶ <https://timeseriesclassification.com/results/ReferenceResults/regression/>.

⁷ <https://www.aeon-toolkit.org/en/latest/examples/benchmarking/benchmarking.html>.

⁸ https://tsml-eval.readthedocs.io/en/latest/examples/regression_experiments.html.

Appendix A: Data description

The list of all 63 datasets in the archive is shown in Table 6. More details on the 44 datasets we have added are given in this section.

Table 6 Time series datasets in the TSER archive

Dataset	Train size	Test size	Length	# Dims	Missing	Category
AppliancesEnergy	95	42	144	24	No	Energy monitoring
HouseholdPowerCon- sumption1	745	686	1440	5	Yes	Energy monitoring
HouseholdPowerCon- sumption2	745	686	1440	5	Yes	Energy monitoring
AustraliaRainfall	112186	48081	24	3	No	Environment monitoring
BeijingPM10Quality	11918	5048	24	9	Yes	Environment monitoring
BeijingPM25Quality	11918	5048	24	9	Yes	Environment monitoring
BenzeneConcentration	3349	5163	240	8	Yes	Environment monitoring
FloodModeling1	471	202	266	1	No	Environment monitoring
FloodModeling2	466	201	266	1	No	Environment monitoring
FloodModeling3	429	184	266	1	No	Environment monitoring
LiveFuelMoistureContent	3493	1510	365	7	No	Environment monitoring
BIDMC32HR	5550	2399	4000	2	No	Health monitoring
BIDMC32RR	5471	2399	4000	2	No	Health monitoring
BIDMC32SpO2	5550	2399	4000	2	No	Health monitoring
Covid3Month	140	61	84	1	No	Health monitoring
IEEEPPG	1768	1328	1000	5	No	Health monitoring
PPGDalia	43215	21482	256-512 ^D	4	No	Health monitoring
NewsHeadlineSentiment	58213	24951	144	3	No	Sentiment analysis
NewsTitleSentiment	58213	24951	144	3	No	Sentiment analysis
DailyOilGasPrices	133	58	30	2	No	Economic analysis
ChilledWaterPredictor	321	138	168	4	No	Energy monitoring
ElectricityPredictor	567	243	168	4	No	Energy monitoring
HotwaterPredictor	245	106	168	4	No	Energy monitoring
OccupancyDetection- Light	237	103	60	3	No	Energy monitoring
SolarRadiationAndalusia	470	202	365	2	Yes	Energy monitoring
SteamPredictor	210	90	168	4	No	Energy monitoring
TetuanEnergyConsump- tion	254	110	144	5	No	Energy monitoring
WindTurbinePower	596	256	144	1	No	Energy monitoring
AcousticContamination- Madrid	166	72	365	1	Yes	Environment monitoring
AluminiumConcentra- tion	440	189	2542	1	No	Environment monitoring
BeijingIntAirportPM- 25Quality	1099	472	24	6	No	Environment monitoring

Table 6 (continued)

Dataset	Train size	Test size	Length	# Dims	Missing	Category
BoronConcentration	438	188	2542	1	No	Environment monitoring
CalciumConcentration	444	191	2307	1	No	Environment monitoring
CopperConcentration	440	189	2542	1	No	Environment monitoring
DailyTemperatureLatitude	27440	11760	365	1	No	Environment monitoring
DhakaHourlyAirQuality	1447	621	24	1	No	Environment monitoring
IronConcentration	427	184	1716	1	No	Environment monitoring
MadridPM10Quality	4845	2077	168	3	Yes	Environment monitoring
MagnesiumConcentration	1560	669	3578	1	No	Environment monitoring
ManganeseConcentration	427	184	1716	1	No	Environment monitoring
MetroInterstateTrafficVolume	849	365	24	4	No	Environment monitoring
ParkingBirmingham	1391	597	14-18 ^S	1	No	Environment monitoring
PhosphorusConcentration	1573	675	3578	1	No	Environment monitoring
PotassiumConcentration	1561	669	3578	1	No	Environment monitoring
PrecipitationAndalusia	470	202	365	4	Yes	Environment monitoring
SierraNevadaMountainsSnow	350	150	30	3	No	Environment monitoring
SodiumConcentration	424	183	1716	1	No	Environment monitoring
SulphurConcentration	444	191	2307	1	No	Environment monitoring
ZincConcentration	445	191	2307	1	No	Environment monitoring
ElectricMotorTemperature	15503	6645	60	6	No	Equipment monitoring
GasSensorArrayAcetone	324	140	7500	1	No	Equipment monitoring
GasSensorArrayEthanol	324	140	7500	1	No	Equipment monitoring
LPGasMonitoringHomeActivity	2017	865	100	1	No	Equipment monitoring
MethaneMonitoring-HomeActivity	2017	865	100	1	No	Equipment monitoring
WaveDataTension	1325	568	57	1	No	Equipment monitoring
BarCrawl6min	140	61	360	3	No	Health monitoring
Covid19Andalusia	142	62	91	1	No	Health monitoring
VentilatorPressure	52815	22635	80	2	No	Health monitoring
BinanceCoinSentiment	184	79	24	2	No	Sentiment analysis
BitcoinSentiment	232	100	24	2	No	Sentiment analysis
CardanoSentiment	74	33	24	2	No	Sentiment analysis
EthereumSentiment	249	107	24	2	No	Sentiment analysis
NaturalGasPricesSentiment	65	28	20	1	No	Sentiment analysis

For those datasets with unequal-length time series: ^S indicates that the unequal-length time series are with respect to the samples, whereas ^D indicates that is with respect to the dimensions

Economic analysis

Oil and natural gas prices

A dataset published on Kaggle⁹ consists of historical prices of Brent Oil, Crude Oil WTI, Natural Gas, and Heating Oil from 2000 to 2022. We created the **DailyOilGasPrices** by using 30 consecutive business days of Crude Oil WTI close prices and traded volumes as predictors and the average natural gas close price during each 30-day time frame as the target variable. The final dataset has 191 2-dimensional time series of length 30, of which 70% were randomly sampled as training data and the remaining 30% as testing data. This type of model could help companies and governments to better analyse and predict economic situations and correlations regarding oil and natural gas.

Energy monitoring

ASHRAE: great energy predictor III

This dataset, published on Kaggle¹⁰, aims to assess the value of energy efficiency improvements. For that purpose, four types of sources are identified: electricity, chilled water, steam and hot water. The goal is to estimate the energy consumption in kWh.

Dimensions correspond to the air temperature, dew temperature, wind direction and wind speed. These values were taken hourly during a week, and the output is the meter reading of the four aforementioned sources. In this way, we created four datasets: **ChilledWaterPredictor**, **ElectricityPredictor**, **HotwaterPredictor**, and **SteamPredictor**. Each dataset has a different number of time series as they correspond to different buildings using those sources. In this sense, **ChilledWaterPredictor** resulted in 459 4-dimensional time series of length 168, **ElectricityPredictor** resulted in 810 4-dimensional time series of length 168, **HotwaterPredictor** resulted in 351 4-dimensional time series of length 168, and **SteamPredictor** resulted in 300 4-dimensional time series of length 168. We randomly sampled 70% of those time series to use as train data and the remaining 30% as test data.

Even though there is a kaggle post indicating that there is one building with meter reading in kBTU they have been transformed into kWh accordingly.

Occupancy detection

In Candanedo and Feldheim (2016), measurements of temperature, light, CO₂, and humidity, collected every minute, were used to detect whether an office room was occupied or not. This data has been made available in the UCI Machine Learning repository¹¹.

⁹ <https://perma.cc/DAP4-JC2A>.

¹⁰ <https://perma.cc/PA63-7GVU>.

¹¹ <https://perma.cc/5ER3-MXG5>.

We created the **OccupancyDetectionLight** dataset by reformulating the problem. We used one hour of temperature (in °C), humidity ratio, and CO₂ concentration (in ppm) as predictors and the average light during that hour (in Lux) as the response variable. This resulted in 340 3-dimensional time series of length 60. We randomly sampled 70% of those time series to use as train data and the remaining 30% as test data. Better models for this data can lead to improvements in energy consumption analysis and prediction.

Solar radiation in Andalusia

This dataset has been obtained from the Andalusia Government (Spain)¹². Data was retrieved from different stations of the 8 districts of Andalusia: Almería, Cádiz, Córdoba, Granada, Huelva, Jaén, Málaga, Sevilla, from 2000 until February 2014. The dataset is known as **SolarRadiationAndalusia**. Dimensions correspond to daily mean of humidity and temperature, whereas the output is the solar radiation for the same day. As time series take daily values during complete years, data from 2014 is not used.

The final dataset includes 672 time series with 2 dimensions with length of 365. The training dataset includes randomly samples 70%, whereas the remaining 30% forms the testing dataset.

Energy consumption in Tetuan

This dataset, published on UCI Machine Learning Repository¹³, aims to estimate the power consumption in three zones in Tetouan Salam and El Hibaoui (2018). The new dataset is known as **TetuanEnergyConsumption**. Data has been collected on a ten minute basis. Hence, time series have 144 values (6 values per hour). A total of 5 dimensions have been identified: temperature, humidity, wind speed, general diffuse flows and diffuse flows. The goal is to estimate the daily average power consumption in the three zones of Tetouan.

The aforementioned dataset includes 364 5-dimensional time series of length 144. 70% of those 364 time series have been randomly selected for the training set, whereas the remaining 30% belong to the testing set.

Wind turbine power generation

The “Wind Turbine Power (kW) Generation Data” dataset on Kaggle¹⁴ consists of large amounts of data collected from a wind turbine, from the temperatures at different parts of the turbines, to the angular position of each blade, and the turbine’s power output. The measurements were made on a 10-minute basis and span from 2019 to 2021.

¹² <https://perma.cc/CH23-UVRJ>.

¹³ <https://perma.cc/NX65-A5B4>.

¹⁴ <https://perma.cc/8X2R-PRUD>.

We used this data to create the **WindTurbinePower** dataset. Each time series consists of 144 timepoints, i.e. one day of measurements, and its target variable is the turbine's average power output on that day. From the 76 possible features, we used only the turbine's torque as a predictor, since it was enough to achieve good results and the other features seemed secondary at best. Some instances were removed due to their respective days having fewer than 144 measurements. The resulting dataset has 852 instances, of which 70% were randomly sampled as training data and the remaining 30% as testing data.

Good regression models for this dataset should improve the logistics of green energy production, by better predicting the energy output of a wind turbine in a given place and/or season.

Environment monitoring

Acoustic contamination in Madrid, Spain

This dataset has been made publicly available by the Government of Madrid, Spain¹⁵. Data is collected by a number of stations located in the city of Madrid. This dataset is updated daily since 2014. However, we created the **AcousticContaminationMadrid** dataset with data up to December 2021. The input time series is the LAeq, a fundamental measurement parameter designed to represent a varying sound source over a given time as a single number. Whereas the output time series is the LAS01, the first percentile of sound pressure levels, with A frequency weighting and slow time weighting, recorded during the corresponding period. Examples of such series and outputs are shown in Fig. 16.

The final dataset includes 238 univariate time series. Moreover, their length is 365, which corresponds to daily values taken during a year. The training dataset is composed of randomly selected 70% of the samples, whereas the remaining 30% composes the testing dataset.

Africa soil chemistry

The Africa Soil Information Service (AfSIS) Soil Chemistry¹⁶ dataset contains large amounts of dry and wet chemistry data obtained from soil samples collected from many countries throughout Sub-Saharan Africa, from 2009 to 2013. Dry chemistry analysis, such as infrared spectroscopy and X-ray fluorescence, is comparably less expensive than wet chemistry. Therefore, a model which uses only dry chemistry to predict certain nutrients measured by wet chemical analyses is commercially interesting.

¹⁵ <https://perma.cc/9V27-BF3F>.

¹⁶ <https://perma.cc/TP5Y-KS6M>.

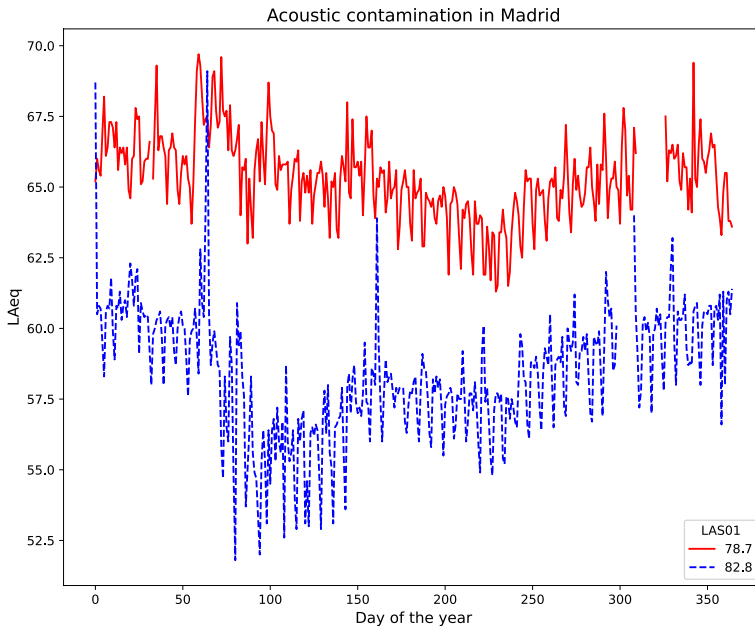


Fig. 16 Two examples of yearly sound pollution in the AcousticContaminationMadrid dataset

The dry chemical measurements were taken using many different machine models, of which we selected four: “Alpha ZnSe”, “Alpha KBr”, “HTSXT” and “MPA”. The wet chemistry data for each soil sample includes the quantity of 12 nutrients: Aluminium, Boron, Copper, Iron, Manganese, Sodium, Phosphorous, Potassium, Magnesium, Sulphur, Zinc and Calcium. Each dry chemical measurement is a time series where the time axis is wavelength and the y-axis is the respective response.

We paired a third of each dry chemistry machine’s experiments to a different nutrient measurement, thus creating 12 datasets. They are **AluminiumConcentration** (629 cases of length 2542), **BoronConcentration** (626 cases of length 2542), **CopperConcentration** (629 cases of length 2542), **IronConcentration** (611 cases of length 1716), **ManganeseConcentration** (611 cases of length 1716), **SodiumConcentration** (607 cases of length 1716), **PhosphorusConcentration** (2248 cases of length 3578), **PotassiumConcentration** (2230 cases of length 3578), **MagnesiumConcentration** (2229 cases of length 3578), **SulphurConcentration** (635 cases of length 2307), **ZincConcentration** (636 cases of length 2307), and **CalciumConcentration** (635 cases of length 2307).

In each dataset, 70% of cases were randomly sampled as training data and the remaining 30% as testing data. Three example soil spectrograms are shown in Fig. 1.

Beijing Airport PM_{2.5} contamination

This dataset was obtained from the UCI Machine Learning Repository¹⁷. Liang et al. (2015) collected hourly data containing the PM_{2.5} data of US Embassy in Beijing, as well as meteorological data from Beijing Capital International Airport. This dataset, known as **BeijingIntAirportPM25Quality**, includes 6-dimensional time series of 24 points. The dimensions are the dew point, temperature, pressure, combined wind direction, and accumulated hours of snow and rain measured, as mentioned, in the Beijing Capital International Airport. The output is the PM_{2.5} data averaged daily.

The aforementioned dataset includes 1571 6-dimensional time series of length 24. 70% of those 1571 time series have been randomly selected for the training set, whereas the remaining 30% belong to the testing set.

Daily temperature and latitude

A dataset published on Kaggle¹⁸ contains daily temperature data for the 1000 most populous cities in the world, along with their geographic coordinates, from 1980 to 2020.

We used this data to create the **DailyTemperatureLatitude** dataset. We split each city's temperature data into 1 year long time series, i.e. 365 timepoints. Leap years were shortened by averaging the temperatures on the 28th and 29th of February. The predictors are the daily temperatures (in °C) recorded during the year and the response variable is the corresponding city's latitude. The final dataset has 39200 univariate time series of length 365, of which 70% were randomly sampled as training data and the remaining 30% as testing data. Two samples of the constructed dataset are shown in Fig. 17.

With exploratory analysis and regression on this dataset, climate change and its effects can be better understood and predicted on a local basis.

Air quality in Dhaka

Data sourced from AirNow¹⁹ and made available on Kaggle²⁰ comprises 7 years of hourly measurements of fine particulate matter (PM_{2.5}) concentrations at the United States Embassy in Dhaka, Bangladesh, along with each corresponding Air Quality Index (AQI).

We used this data to create the **DhakaHourlyAirQuality** dataset, in which the predictors are 24 hours of PM_{2.5} concentrations and the response variable is the average AQI on that respective day. Thus, better models can be created and tested to more cheaply and quickly evaluate air quality in different cities. The resulting

¹⁷ <https://perma.cc/G8VS-DKKY>.

¹⁸ <https://perma.cc/3KPY-YHW2>.

¹⁹ <https://perma.cc/2K2D-LT6T>.

²⁰ <https://perma.cc/7865-ZAAD>.

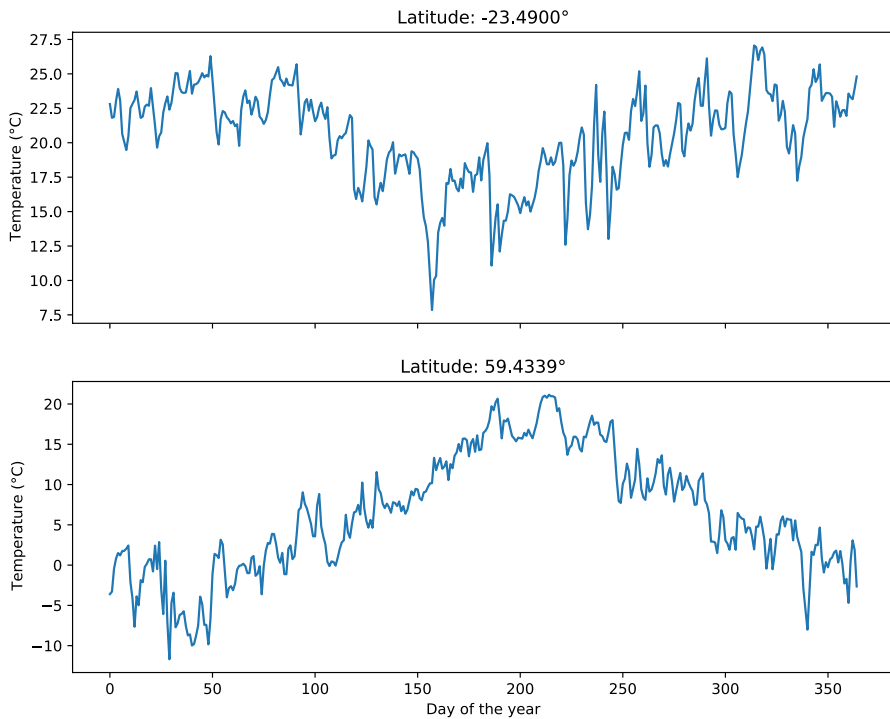


Fig. 17 Examples of yearly temperature profiles from two different latitudes in the DailyTemperatureLatitude dataset

dataset consists of 2068 univariate time series of length 24, of which 70% were randomly sampled as training data and the remaining 30% as testing data.

Madrid PM10 contamination

This dataset is publicly available on Kaggle²¹, even though the data was originally published in the public repository of the Madrid Government²². The dataset is known as **MadridPM10Quality**. Its input data consists in measurements of the level of sulphur dioxide, carbon monoxide and nitric oxide. Time series correspond to hourly values measured during a week. The output value is the weekly averaged PM_{10} .

The final dataset includes 6922 3-dimensional time series with a length of 168. The training dataset is composed of randomly selected 70% of the samples, whereas the remaining 30% forms the testing dataset.

²¹ <https://perma.cc/ZUS5-E26E>.

²² <https://perma.cc/5QNM-ZWGS>.

Metro Interstate Traffic Volume

This dataset is publicly available on the UCI Machine Learning Repository²³. Data consists of hourly traffic volume for MN DoT ATR station 301, roughly midway between Minneapolis and St Paul, collected from 2012 to 2018. The dataset, known as **MetroInterstateTrafficVolume**, aims to estimate the daily average traffic volume for the aforementioned road. The input dimensions consists of the average temperature, the amount of rain and snow, and the percentage of cloud cover.

The final dataset includes 1214 time series with 4 dimensions. Moreover, their length is 24, which corresponds to hourly measures taken during a day. The training dataset includes a randomly selected 70% of the whole data, whereas the remaining 30% forms the testing dataset.

Parking Birmingham

This dataset is publicly available on the UCI Machine Learning Repository²⁴. Stolfi et al. (2017) collected data from car parks in Birmingham (United Kingdom) operated by National Car Parks from Birmingham City Council. The dataset, known as **ParkingBirmingham**, aims to estimate occupancy rates from 2016/10/04 to 2016/12/19. Input time series is the number of parked cars every hour, whereas the output is the occupancy rate. The total number of hours measured per day varies from 14 to 18.

The aforementioned dataset includes 1988 unequal length time series (with lengths between 14 and 18). 70% of those 1988 time series have been randomly selected for the training set, whereas the remaining 30% belong to the testing set.

Precipitation in Andalusia

This dataset has been obtained from the Andalusia Government (Spain)²⁵. Data was retrieved from different stations in the 8 districts of Andalusia: Almeria, Cadiz, Cordoba, Granada, Huelva, Jaen, Malaga, and Sevilla, from 2000 until February 2014. This dataset, known as **PrecipitationAndalusia**, includes the daily averaged temperature, humidity, and wind speed and direction as inputs, whereas the output is the average precipitation. Two examples are illustrated in Fig. 18.

This resulted in 672 4-dimensional time series of length 365. We randomly sampled 70% of those time series to use as train data and the remaining 30% as test data.

²³ <https://perma.cc/B6FV-SLCG>.

²⁴ <https://perma.cc/3LAJ-YTSG>.

²⁵ <https://perma.cc/3APP-2L43>.

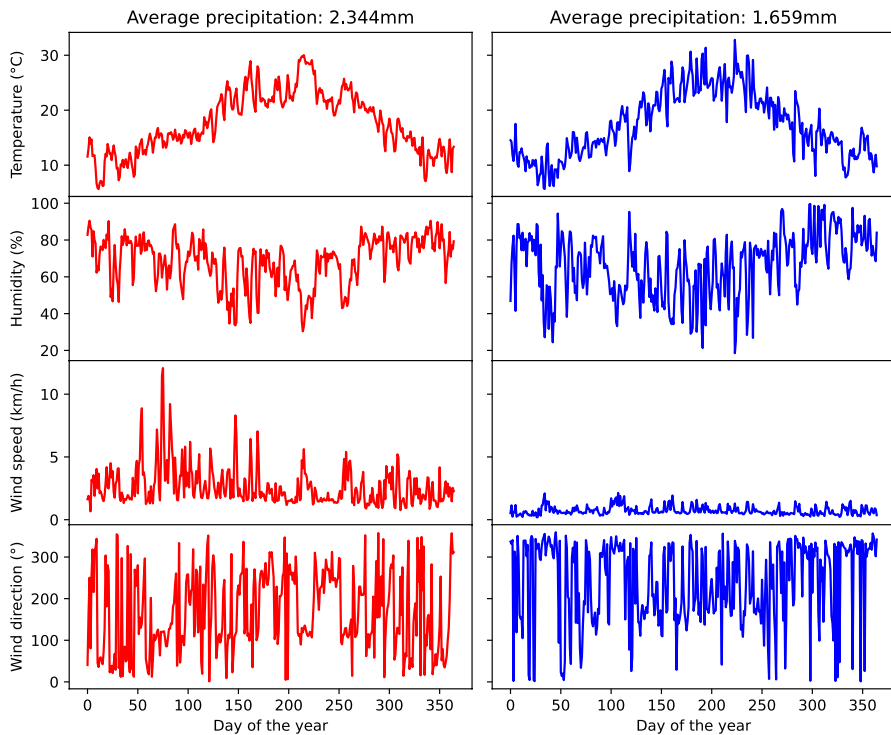


Fig. 18 Two different years of measurements and the associated average precipitation in the PrecipitationAndalusia dataset

Sierra Nevada mountains measurements

A dataset published in Osterhuber and Schwartz (2021) and made available on Kaggle²⁶ consists of daily measurements of minimum and maximum air temperatures, precipitation, and snowpack characteristics made at a field station in Sierra Nevada, United States, between 1971 and 2019.

We split the measurements into groups of 30 consecutive days and used the minimum and maximum air temperature (in °C) and precipitation (in mm) to create 3-dimensional time series of length 30. Using the amount of new snow (in cm) accumulated during each respective 30-day timeframe as the target variable, we created the **SierraNevadaMountainsSnow** dataset, with 500 instances in total. Around 20 instances were removed due to missing values. The dataset is split into train and test sets by randomly sampling 30% of the data as test. This data can be used to train models to predict heavy snowfall and prepare cities and roads for harsh weather conditions.

²⁶ <https://perma.cc/2KQT-C5AW>.

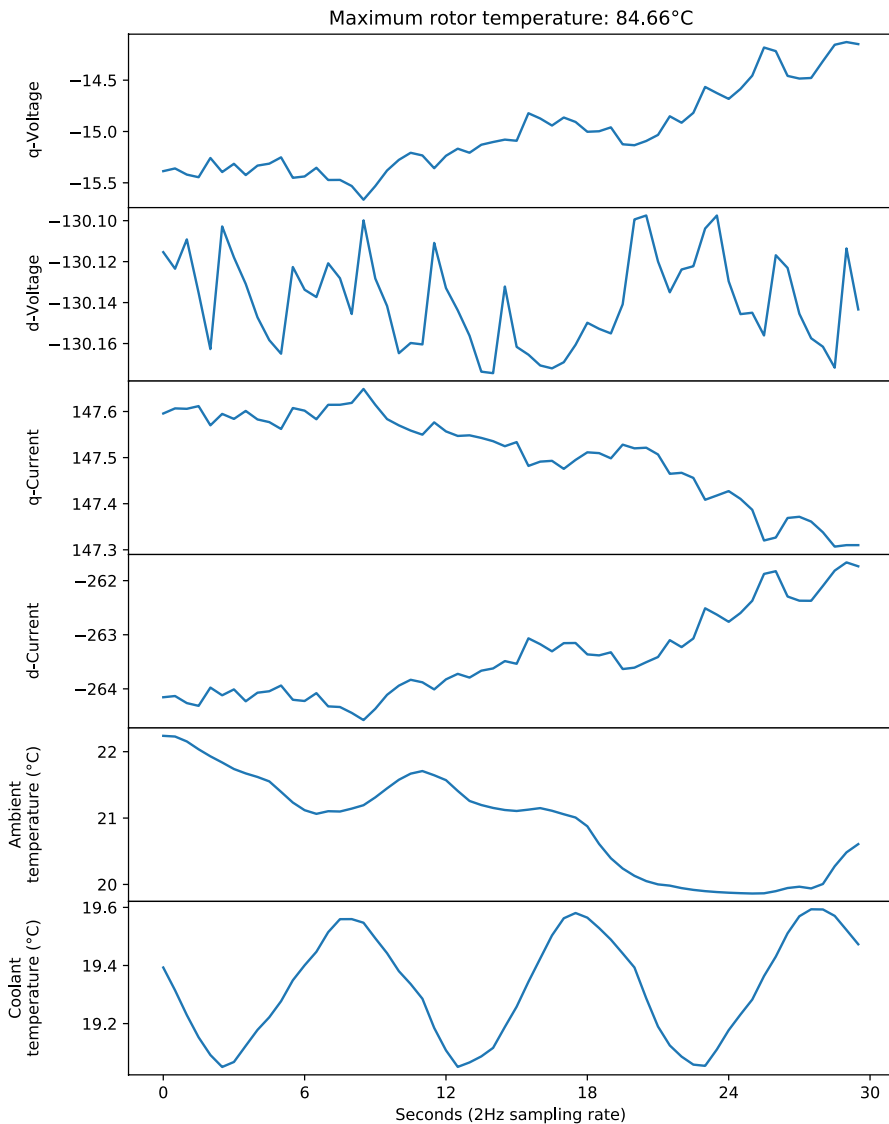


Fig. 19 Example of currents, voltages, and ambient and coolant temperatures measurements used to predict maximum rotor temperature in the ElectricMotorTemperature dataset

Equipment monitoring

Electric motor temperature

The authors Kirchgässner et al. (2021) collected large amounts of sensor data, made available on Kaggle²⁷, from a permanent magnet synchronous motor

²⁷ <https://perma.cc/A7FG-KFLT>.

(PMSM) deployed on a test bench. The dataset consists of multiple measurement sessions, each ranging between one and six hour long, whose recordings were sampled at 2Hz. The sensors collected a variety of features, such as current and voltage, ambient, coolant, and rotor temperatures, and motor speed. Rotor temperature, specifically, is not reliably and economically measurable in a commercial vehicle, thus being an interesting candidate for response variable. The data is, therefore, useful for industrial processes and monitoring.

Therefore, we created the **ElectricMotorTemperature** dataset by first splitting the measurement sessions into groups of 30 consecutive seconds, i.e. 60 timepoints. Then, we used the recorded ambient and coolant temperatures and d and q components of voltage and current as predictors to form 6-dimensional time series of length 60. The target variable is the maximum recorded rotor temperature during each respective 30-second time frame. An example of such a time series is shown in Fig. 19. The resulting dataset has 22148 instances, of which 70% were sampled as training data and the remaining 30% as testing data.

Home activity monitoring of gases

This dataset has been obtained from UCI Machine Learning Repository²⁸. The authors Huerta et al. (2016) collected recordings of a gas sensor array composed of 8 MOX gas sensors, as well as a temperature and humidity sensor. Sensors were exposed to three different conditions: presence of wine, banana and background activity. Two datasets have been collated from this repository: **LPGasMonitoringHomeActivity** and **MethaneMonitoringHomeActivity**. The first estimates the liquefied petroleum gas concentration from humidity measurements. On the other hand, the second one estimates the methane concentration from temperature measurements. The latter is illustrated in Fig. 20.

This resulted in 2882 univariate time series of length 100. We randomly sampled 70% of those time series to use as train data and the remaining 30% as test data.

Gas sensor array under flow modulation

Ziyatdinov et al. (2015) combined an array of 16 metal-oxide gas sensors and an external mechanical ventilator to simulate sniffing behaviour within the biological respiration cycle. The study extracted high and low frequency features from the signals and proposed a regression problem where the predictors are either of those features and the responses are the concentrations of two analytes used to form test gasses, acetone and ethanol. The data collected by the authors has been made available in the UCI Machine Learning repository²⁹. The development of better regression techniques for this data should lead to improvements in early detection of gases in chemo-sensory systems.

²⁸ <https://perma.cc/69GV-CR7S>.

²⁹ <https://perma.cc/WD2S-KEV9>.

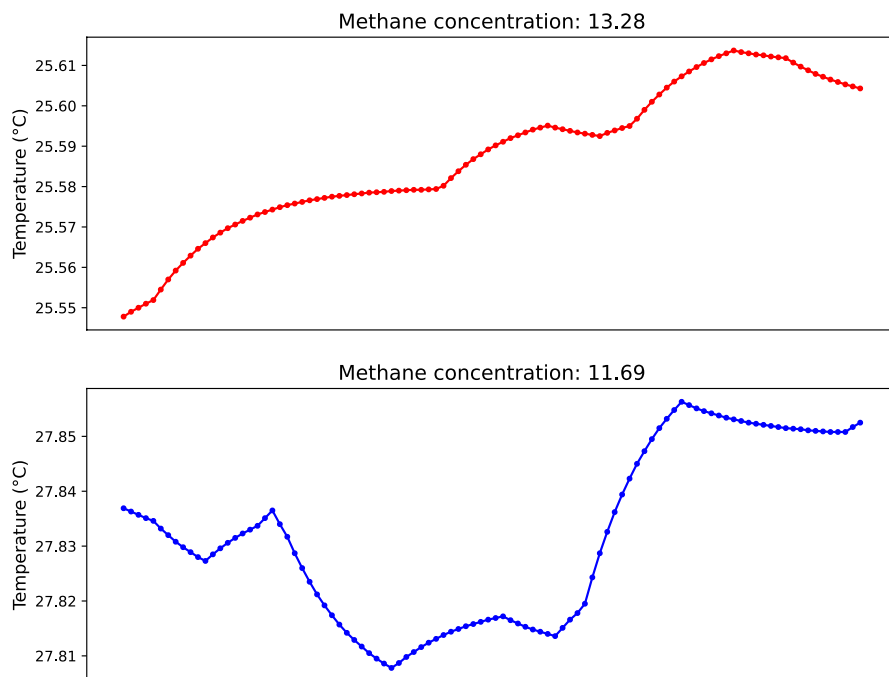


Fig. 20 Temperature data and respective methane concentration in the MethaneMonitoringHomeActivity dataset

We used the raw signal data with 58 samples, each one consisting of 16 sensors and totalling 928 time series, to create two datasets: **GasSensorArrayEthanol**, in which the target variable is the ethanol concentration in the tested gas, and **GasSensorArrayAcetone**, in which the target is similarly the concentration of acetone. Both consist of 464 univariate time series of length 7500 and are split into train and test sets by randomly selecting 30% of instances as test.

Wave elevation and line tension

A dataset published on Kaggle³⁰ consists of a simulation of a ship. The dataset's goal is to predict the tension of a string given temporal wave elevation data. While the data is simulated, well-fitted models should improve monitoring of naval equipment during harsh conditions.

Thus, we created the **WaveTensionData** dataset by separating the source data into univariate time series of length 57 and using wave height as the predictor and the corresponding average string tension as the target variable. The resulting dataset

³⁰ <https://perma.cc/BAG4-W8SL>.

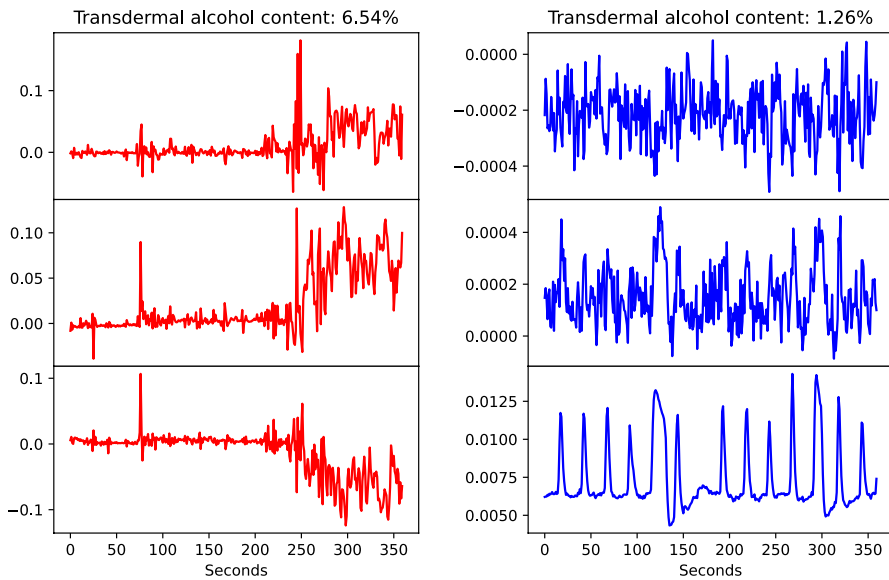


Fig. 21 Two samples of accelerometer data and alcohol concentration in the BarCrawl6min dataset

has 1893 instances, of which 70% were sampled as training data and the remaining 30% as testing data.

Health monitoring

Bar Crawl: detecting heavy drinking episodes

This dataset, made available in the UCI Machine Learning repository³¹ by their authors Killian et al. (2019), consists in predicting heavy drinking episodes via mobile data. Data was collected from smartphones from 13 participants. The goal is to estimate the transdermal alcohol content by using an accelerometer. We have, therefore, created the **BarCrawl6min** dataset, where each input dimension corresponds to a different axis of the accelerometer. Moreover, even though data was recorded at 30 minutes intervals, in order to accurately estimate the drinking episode, only the last 6 minutes from the recording were kept. Two resulting samples are shown in Fig. 21. Note that all data is fully anonymised and that TAC readings were preprocessed/cleaned by the authors.

This resulted in 201 3-dimensional time series of length 360, which correspond to 6 minutes of secondly measurements. We randomly sampled 70% of those time series to use as train data and the remaining 30% as test data.

³¹ <https://perma.cc/7VP5-79LN>.

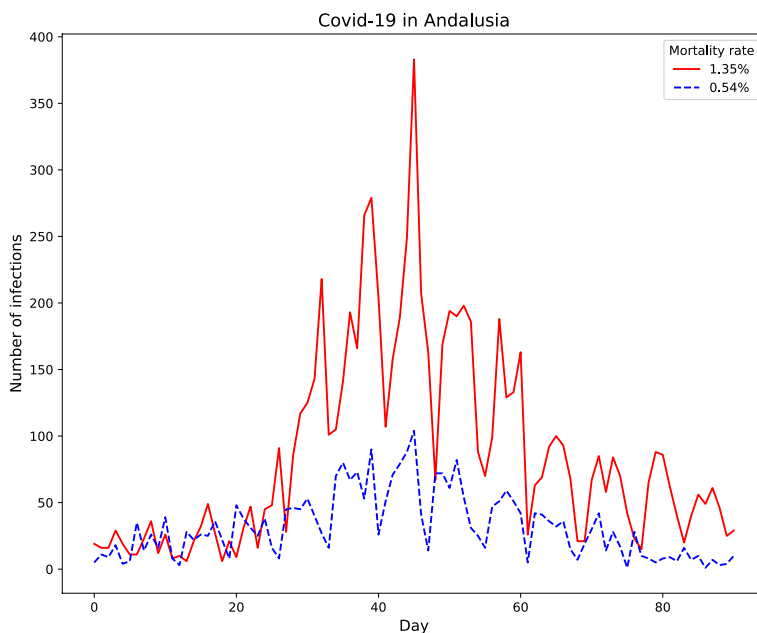


Fig. 22 Two different Covid-19 waves and their mortality rates in the Covid19Andalusia dataset

Covid-19 in Andalusia

This dataset consists of estimating the mortality rate during Covid-19 waves and in different districts in eight different areas of Andalusia, its country's second largest and most populated autonomous region, located in southern Spain. The output is the number of deaths in proportion to the total number of infected people in that district. This dataset, known as **Covid19Andalusia**, has been made public by the authors Díaz-Lozano et al. (2022), who took the data available in the Andalusia Government Website³². This dataset comprises 6 waves from a total of 34 districts. All waves are equal-length (91 points) since we have considered 45 days before and after the peak of the outbreak, as it has been demonstrated to be the most relevant data. Two such waves are illustrated in Fig. 22.

The aforementioned dataset includes 204 unidimensional time series of length 91. 70% of those 204 time series have been randomly selected for the training set, whereas the remaining 30% belong to the testing set.

Pressure of a ventilator connected to a sedated patient's lung

This dataset was made available by Kaggle in collaboration with Google Brain³³. Data used in this competition was produced by connecting a ventilator to an artificial

³² <https://perma.cc/U55Q-U4KZ>.

³³ <https://perma.cc/RQW7-QH7L>.

bellows test lung through a respiratory circuit. The goal is to estimate the pressure, a value ranging from 0 to 100, representing how much the inspiratory solenoid valve is open to let air into the lung. The proposed dataset, named **VentilatorPressure**, includes time series corresponding to approximately 3-second breaths. Dimensions are the control input and output for the inspiratory solenoid valve.

The final dataset includes 75450 time series with 2 dimensions and length 80. The training dataset comprises randomly sampled 70% of the series, whereas the remaining 30% belong to the testing dataset.

Sentiment analysis

Cryptocurrency sentiment

By combining historical sentiment data for 4 cryptocurrencies, extracted from EODHistoricalData³⁴ and made available on Kaggle³⁵, with historical price data for the same cryptocurrencies, extracted from CryptoDataDownload³⁶, we created the **BitcoinSentiment**, **EthereumSentiment**, **CardanoSentiment**, and **BinanceCoinSentiment** datasets, with 332, 356, 107, and 263 total instances, respectively.

In all four datasets, the predictors are hourly close price (in USD) and traded volume for each respective cryptocurrency during a day, resulting in 2-dimensional time series of length 24. The response variable is the normalized sentiment score on the day spanned by the timepoints. The datasets were split into train and test sets by randomly selecting 30% of each set as test data. Using this data, companies can better prepare for shifts of public perception regarding cryptocurrencies.

Sentiment on natural gas prices

Natural gas prices historical data was taken from the U.S. Energy Information Administration³⁷ along with corresponding sentiment scores obtained by analysing relevant tweets on the topic. From this data, we created the **NaturalGasPricesSentiment** dataset.

We first split the data into groups of 20 consecutive business days. We then used the daily natural gas prices as predictors and the average sentiment score during each 20-day time frame as the response variable. The final dataset has 93 univariate time series of length 20, of which 70% were randomly sampled as training data and the remaining 30% as testing data. Two of those time series are shown in Fig. 23. Again, companies and local governments can use the data to analyse and predict shifts in public perception on natural gas.

³⁴ <https://perma.cc/37GN-BMRL>.

³⁵ <https://perma.cc/J6LK-99Q5>.

³⁶ <https://perma.cc/4M79-7QY4>.

³⁷ <https://perma.cc/8AP5-5R7R>.

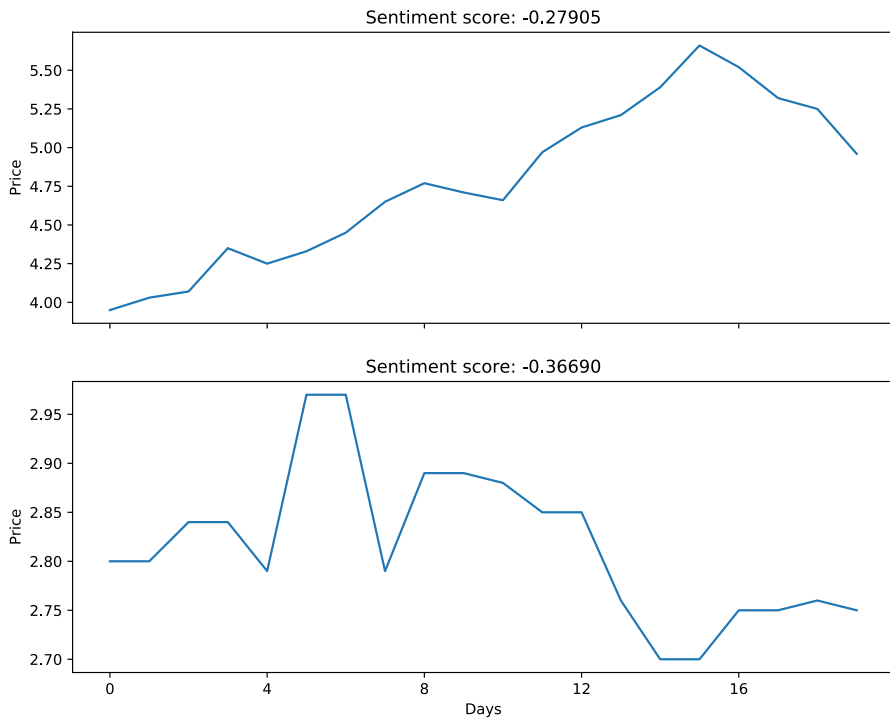


Fig. 23 Examples of gas price time series from the NaturalGasPriceSentiment dataset and the respective sentiment score

Regressor configurations

Parameter settings for all algorithms are shown in Table 7.

Table 7 Regressor configurations for our experiments where m is the series length, d is the number of dimensions and rm is the lengths of DrCIF representations

Regressor	Configuration
Ridge	Regularization strength: $\in \{10^{-3}, 10^{-2}, \dots, 10^3\}$
grid-SVR	Kernel $\in \{\text{RBF}, \text{Sigmoid}\}$
	Regulatisation parameter $\in \{10^{-1}, 10^0, \dots, 10^2\}$
	Kernel Coefficient $\in \{10^{-3}, 10^{-2}, \dots, 10^0\}$
RandF	Num. Estimators: 500
RotF	Num. Estimators: 500
XGBoost	Num. Estimators: 500
	Learning Rate: 0.1
FPCR	FPCA Num. Components: 10

Table 7 (continued)

Regressor	Configuration
FPCR-Bs	Smooth: B-splines FPCA Num. Components: 10 Num. Basis Functions: 10 Order: 4
ROCKET	Num. Kernels: 10000
MultiROCKET	Num. Kernels: 6250 Max. Num. Dilations per Kernel: 32 Num. Features per Kernel: 4
1NN-ED	Num. Neighbours: 1 5
5NN-ED	Distance measure: Euclidean
1NN-DTW	Num. Neighbours: 1 5
5NN-DTW	Distance measure: DTW
FreshPRINCE	Num. Estimators: 500
TSF	Num. Estimators: 500 Num. Intervals per Tree: $\sqrt{m}$
DrCIF	Num. Estimators: 500 Num. Intervals per Representation: $4 + (\sqrt{d}\sqrt{rm})/3$ Num. Features per Tree: 10
CNN	Num. Epochs: 2000 Batch size: 16 Kernel size: 7 Num. Convolutional Layers: 2
FCN	Num. Epochs: 2000 Batch size: 16 Num. Components: 1 (InceptionE: 5)
Inception	Num. Epochs: 1500
InceptionE	Batch size: 64 Kernel size: 40
ResNet	Num. Epochs: 1500 Batch size: 16

Acknowledgements This work has been partially supported by “Agencia Española de Investigación (España)” (grant ref.: PID2020-115454GB-C22 / AEI / 10.13039 / 501100011033). David Guijo-Rubio’s research has been supported by the University of Córdoba through grants to Public Universities for the requalification of the Spanish university system of the Ministry of Universities, financed by the European Union - NextGenerationEU (grant reference: UCOR01MS). Anthony Bagnall and Matthew Middlehurst’s contribution was funded by the UK Engineering and Physical Sciences Research Council (grant reference EP/W030756/1). Guilherme Arcencio’s research has been funded by the São Paulo Research Foundation (FAPESP) (grant refs.: #2022/12486-4 and #2022/00305-5). Diego Furtado Silva’s research has also been supported by FAPESP (grant ref.: #2022/03176-1). Some of the experiments were carried out on the High Performance Computing Cluster supported by the Research and Specialist Computing Support service at the University of East Anglia. We would like to thank all those responsible for helping maintain the time series machine learning archives and those contributing to open source implementations of the algorithms.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Bagnall A, Lines J, Bostrom A et al (2017) The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Min Knowl Disc* 31(3):606–660. <https://doi.org/10.1007/s10618-016-0483-9>
- Bagnall A, Flynn M, Large J, et al (2018) Is rotation forest the best classifier for problems with continuous features? [arXiv:1809.06705](https://arxiv.org/abs/1809.06705)<https://doi.org/10.48550/arXiv.1809.06705>
- Bayani M (2022) Essays on machine learning methods in economics. Phd thesis, City University of New York, https://academicworks.cuny.edu/cgi/viewcontent.cgi?article=6069&context=gc_etds
- Benavoli A, Corani G, Mangili F (2016) Should we really use post-hoc tests based on mean-ranks? *J Mach Learn Res* 17:1–10
- Bloemheuveld S, van den Hoogen J, Jozinović D et al (2022) Graph neural networks for multivariate time series regression with application to seismic data. *Int J Data Sci Anal*. <https://doi.org/10.1007/s41060-022-00349-6>
- Bostrom A, Bagnall A (2017) Binary shapelet transform for multiclass time series classification. *Trans Large-Scale Data Knowl Center Syst* 32:24–46. https://doi.org/10.1007/978-3-662-55608-5_2
- Breiman L (2001) Random forests. *Mach Learn* 45(1):5–32. <https://doi.org/10.1023/A:1010933404324>
- Breiman L (2017) Classification and regression trees. Routledge, Boca Raton
- Cabello N, Naghizade E, Qi J, et al (2020) Fast and accurate time series classification through supervised interval search. In: 2020 IEEE international conference on data mining (ICDM), IEEE, pp 948–953, <https://doi.org/10.1109/icdm50108.2020.00107>
- Candanedo LM, Feldheim V (2016) Accurate occupancy detection of an office room from light, temperature, humidity and CO₂ measurements using statistical learning models. *Energy Build* 112:28–39. <https://doi.org/10.2139/ssrn.3686755>
- Chen T, Guestrin C (2016) Xgboost: a scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 785–794, <https://doi.org/10.1145/2939672.2939785>
- Christ M, Braun N, Neuffer J et al (2018) Time series feature extraction on basis of scalable hypothesis tests (tsfresh-a python package). *Neurocomputing* 307:72–77. <https://doi.org/10.1016/j.neucom.2018.03.067>
- Dempster A, Petitjean F, Webb GI (2020) Rocket: exceptionally fast and accurate time series classification using random convolutional kernels. *Data Min Knowl Disc* 34(5):1454–1495. <https://doi.org/10.1007/s10618-020-00701-z>
- Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. *J Mach Learn Res* 7:1–30. <https://doi.org/10.5555/1248547.1248548>
- Deng H, Runger G, Tuv E et al (2013) A time series forest for classification and feature extraction. *Inf Sci* 239:142–153. <https://doi.org/10.1016/j.ins.2013.02.030>
- Díaz-Lozano M, Guijo-Rubio D, Gutiérrez PA et al (2022) Covid-19 contagion forecasting framework based on curve decomposition and evolutionary artificial neural networks: A case study in andalusia, spain. *Expert Syst Appl* 207:117977. <https://doi.org/10.1016/j.eswa.2022.117977>
- Drucker H, Burges CJ, Kaufman L, et al (1996) Support vector regression machines. *Adv Neural Inf Process Syst*. <https://proceedings.neurips.cc/paper/1996/file/d38901788c533e8286cb6400b40b386d-Paper.pdf>

- Flynn M, Large J, Bagnall A (2019) The contract random interval spectral ensemble (c-rise): the effect of contracting a classifier on accuracy. In: International conference on hybrid artificial intelligence systems. Springer, pp 381–392, https://doi.org/10.1007/978-3-030-29859-3_33
- Fulcher BD, Jones NS (2017) HCTSA: a computational framework for automated time-series phenotyping using massive feature extraction. *Cell Syst* 5(5):527–531. <https://doi.org/10.1016/j.cels.2017.10.001>
- García S, Herrera F (2008) An extension on “statistical comparisons of classifiers over multiple data sets” for all pairwise comparisons. *J Mach Learn Res* 9:2677–2694
- Ghosh R (2022) Natural gas prices with Twitter sentiment scores. <https://doi.org/10.34740/KAGGLE/DSV/3953184>
- Goldsmith J, Scheipl F (2014) Estimator selection and combination in scalar-on-function regression. *Comput Stat Data Anal* 70:362–372. <https://doi.org/10.1016/j.csda.2013.10.009>
- Gu A, Johnson I, Goel K, et al (2021) Combining recurrent, convolutional, and continuous-time models with linear state space layers. In: Ranzato M, Beygelzimer A, Dauphin Y, et al (eds) Advances in neural information processing systems, vol 34. Curran Associates, Inc., pp 572–585, <https://proceedings.neurips.cc/paper/2021/file/05546b0e38ab9175cd905eebcc6ebb76-Paper.pdf>
- Hasani R, Lechner M, Wang TH, et al (2022) Liquid structural state-space models. *arXiv:2209.12951*
- He K, Zhang X, Ren S, et al (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 770–778, <https://doi.org/10.1109/cvpr.2016.90>
- Huerta R, Mosqueiro T, Fonollosa J et al (2016) Online decorrelation of humidity and temperature in chemical sensors for continuous monitoring. *Chemom Intell Lab Syst* 157:169–176. <https://doi.org/10.1016/j.chemolab.2016.07.004>
- Ismail Fawaz H, Forestier G, Weber J et al (2019) Deep learning for time series classification: a review. *Data Min Knowl Disc* 33(4):917–963. <https://doi.org/10.1007/s10618-019-00619-1>
- Ismail Fawaz H, Lucas B, Forestier G et al (2020) Inceptiontime: finding alexnet for time series classification. *Data Min Knowl Disc* 34(6):1936–1962. <https://doi.org/10.1007/s10618-020-00710-y>
- Keogh EJ, Pazzani MJ (2001) Derivative dynamic time warping. In: Proceedings of the 2001 SIAM international conference on data mining, SIAM, pp 1–11, <https://doi.org/10.1137/1.9781611972719.1>
- Killian JA, Passino KM, Nandi A, et al (2019) Learning to detect heavy drinking episodes using smartphone accelerometer data. In: KHD@ IJCAI, pp 35–42, <https://ceur-ws.org/Vol-2429/paper6.pdf>
- Kirchgässner W, Wallscheid O, Böcker J (2021) Estimating electric motor temperatures with deep residual machine learning. *IEEE Trans Power Electron* 36(7):7480–7488. <https://doi.org/10.1109/tpel.2020.3045596>
- Liang X, Zou T, Guo B et al (2015) Assessing Beijing’s pm2.5 pollution: severity, weather impact, APEC and winter heating. *Proc R Soc A Math Phys Eng Sci* 471(2182):20150257. <https://doi.org/10.1098/rspa.2015.0257>
- Lubba CH, Sethi SS, Knaute P et al (2019) catch22: Canonical time-series characteristics. *Data Min Knowl Disc* 33(6):1821–1852. <https://doi.org/10.1007/s10618-019-00647-x>
- Makridakis S, Wheelwright SC, Hyndman RJ (2008) Forecasting methods and applications. Wiley, New York
- Middlehurst M, Bagnall A (2022) The freshprince: a simple transformation based pipeline time series classifier. In: International conference on pattern recognition and artificial intelligence. Springer, pp 150–161, https://doi.org/10.1007/978-3-031-09282-4_13
- Middlehurst M, Large J, Bagnall A (2020a) The canonical interval forest (CIF) classifier for time series classification. In: 2020 IEEE international conference on big data (big data), IEEE, pp 188–195, <https://doi.org/10.1109/bigdata50022.2020.9378424>
- Middlehurst M, Large J, Cawley G, et al (2020b) The temporal dictionary ensemble (TDE) classifier for time series classification. In: Proceedings of European conference on machine learning and principles and practice of knowledge discovery in databases, pp 660–676, https://doi.org/10.1007/978-3-030-67658-2_38
- Middlehurst M, Large J, Flynn M et al (2021) Hive-cote 2.0: a new meta ensemble for time series classification. *Mach Learn* 110(11):3211–3243. <https://doi.org/10.1007/s10994-021-06057-9>
- Middlehurst M, Schäfer P, Bagnall A (2023) Bake off redux: a review and experimental evaluation of recent time series classification algorithms. *arXiv:2304.13029*
- Pardo C, Diez-Pastor JF, García-Osorio C et al (2013) Rotation forests for regression. *Appl Math Comput* 219(19):9914–9924. <https://doi.org/10.1016/j.amc.2013.03.139>

- Osterhuber R, Schwartz A (2021) Snowpack, precipitation, and temperature measurements at the Central Sierra Snow Laboratory for water years 1971 to 2019. <https://doi.org/10.6078/D1941T>
- Reiss PT, Goldsmith J, Shang HL et al (2017) Methods for scalar-on-function regression. *Int Stat Rev* 85(2):228–249. <https://doi.org/10.1111/insr.12163>
- Rodriguez JJ, Kuncheva LI, Alonso CJ (2006) Rotation forest: a new classifier ensemble method. *IEEE Trans Pattern Anal Mach Intell* 28(10):1619–1630. <https://doi.org/10.1109/tpami.2006.211>
- Ruiz AP, Flynn M, Large J et al (2021) The great multivariate time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Min Knowl Disc* 35(2):401–449. <https://doi.org/10.1007/s10618-020-00727-3>
- Salam A, El Hibaoui A (2018) Comparison of machine learning algorithms for the power consumption prediction:-case study of tetouan city. In: 2018 6th international renewable and sustainable energy conference (IRSEC), IEEE, pp 1–5. <https://doi.org/10.1109/irsec.2018.8703007>
- Schäfer P (2015) The BOSS is concerned with time series classification in the presence of noise. *Data Min Knowl Disc* 29(6):1505–1530. <https://doi.org/10.1007/s10618-014-0377-7>
- Schäfer P, Leser U (2023) Weasel 2.0-a random dilated dictionary transform for fast, accurate and memory constrained time series classification. *Mach Learn*. <https://doi.org/10.1007/s10994-023-06395-w>
- Shokoohi-Yekta M, Hu B, Jin H et al (2017) Generalizing DTW to the multi-dimensional case requires an adaptive approach. *Data Min Knowl Disc* 31(1):1–31. <https://doi.org/10.1007/s10618-016-0455-0>
- Siddiquee MA, Souza VMA, Baker GE, et al (2022) Septor: Seismic depth estimation using hierarchical neural networks. In: Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining, pp 3889–3897. <https://doi.org/10.1145/3534678.3539166>
- Stolfi DH, Alba E, Yao X (2017) Predicting car park occupancy rates in smart cities. In: Smart cities: second international conference, smart-CT 2017, Springer, pp 107–117. https://doi.org/10.1007/978-3-319-59513-9_11
- Tan CW, Bergmeir C, Petitjean F et al (2021) Time series extrinsic regression. *Data Min Knowl Disc* 35(3):1032–1060. <https://doi.org/10.1007/s10618-021-00745-9>
- Tan CW, Dempster A, Bergmeir C et al (2022) Multirocket: multiple pooling operators and transformations for fast and effective time series classification. *Data Min Knowl Disc* 36(5):1623–1646. <https://doi.org/10.1007/s10618-022-00844-1>
- Wang Z, Yan W, Oates T (2017) Time series classification from scratch with deep neural networks: a strong baseline. In: Proceedings of the IEEE international joint conference on neural networks, pp 1578–1585. <https://doi.org/10.48550/arXiv.1611.06455>
- Ye L, Keogh E (2011) Time series shapelets: a novel technique that allows accurate, interpretable and fast classification. *Data Min Knowl Disc* 22(1–2):149–182. <https://doi.org/10.1007/s10618-010-0179-5>
- Yeo I, Johnson RA (2000) A new family of power transformations to improve normality or symmetry. *Biometrika* 87(4):954–959. <https://doi.org/10.1093/biomet/87.4.954>
- Zhao B, Lu H, Chen S et al (2017) Convolutional neural networks for time series classification. *J Syst Eng Electron* 28(1):162–169. <https://doi.org/10.21629/JSEE.2017.01.18>
- Ziyatdinov A, Fonollosa J, Fernández L et al (2015) Bioinspired early detection through gas flow modulation in chemo-sensory systems. *Sens Actuators B Chem* 206:538–547. <https://doi.org/10.1016/j.snb.2014.09.001>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

David Guijo-Rubio^{1,2} · Matthew Middlehurst³ · Guilherme Arcencio⁴ ·
Diego Furtado Silva⁵ · Anthony Bagnall^{2,3}

✉ Anthony Bagnall
a.j.bagnall@soton.ac.uk

David Guijo-Rubio
dguijo@uco.es

Matthew Middlehurst
m.b.middlehurst@soton.ac.uk

Guilherme Arcencio
garcencio@estudante.ufscar.br

Diego Furtado Silva
diegofsilva@usp.br

- ¹ Department of Computer Science and Numerical Analysis, University of Córdoba, Córdoba, Spain
- ² School of Computing Sciences, University of East Anglia, Norwich, UK
- ³ School of Electronics and Computer Science, University of Southampton, Southampton, UK
- ⁴ Department of Computing, Universidade Federal de São Carlos, São Carlos, Brazil
- ⁵ Institute of Mathematics and Computer Sciences, Universidade de São Paulo, São Carlos, Brazil

© The Author(s) 2024. This work is published under

<http://creativecommons.org/licenses/by/4.0/>

(the “License”). Notwithstanding the ProQuest Terms and Conditions, you may use this content in accordance with the terms of the License.